

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int n=sc.nextInt();
6         int[]a=new int[n];
7         for(int i=0;i<n;i++){
8             a[i]=sc.nextInt();
9         }
10        int pivot=sc.nextInt();
11        ArrayList<Integer>l1=new ArrayList<>();
12        ArrayList<Integer>l2=new ArrayList<>();
13        //ArrayList<Integer>l3=new ArrayList<>();
14        for(int num:a){
15            if(num<pivot){
16                l1.add(num);
17            }
18            else if(num>pivot){
19                l2.add(num);
20            }
21        }
22        Collections.sort(l2,Collections.reverseOrder());
23        Collections.sort(l1,Collections.reverseOrder());
24        for(int nu:l1){
25            System.out.print(nu+" ");
26        }
27        for(int nuuu:a){
28            if(nuuu==pivot){
29                System.out.print(nuuu+" ");
30            }
31        }
32        for(int nuu:l2){
33            System.out.print(nuu+" ");
34        }
35    }
36 }
37 }
```

Time left 1:29:47

Hide

There are n concert tickets available, each with a certain price. Then, m customers arrive, one after another.

Each customer announces the maximum price they are willing to pay for a ticket, and after this, they will get a ticket with the nearest possible price such that it does not exceed the maximum price.

Input

The first input line contains integers n and m : the number of tickets and the number of customers.

The next line contains n integers $h_1, h_2, \dots, h_n$: the price of each ticket.

The last line contains m integers $t_1, t_2, \dots, t_m$: the maximum price for each customer in the order they arrive.

Output

Print, for each customer, the price that they will pay for their ticket. After this, the ticket cannot be purchased again.

If a customer cannot get any ticket, print -1 .

Constraints

- $1 \leq n, m \leq 2 \cdot 10^5$
- $1 \leq h_i, t_i \leq 10^9$

Example

Input:

```
5 3
5 3 7 8 5
4 8 3
```

Output:

```
3
8
-1
```

Language java

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt(); // number of tickets
8         int m = sc.nextInt(); // number of customers
9
10        TreeMap<Integer, Integer> map = new TreeMap<>();
11
12        // read ticket prices
13        for (int i = 0; i < n; i++) {
14            int price = sc.nextInt();
15            map.put(price, map.getDefault(price, 0) + 1);
16        }
17
18        // process each customer
19        for (int i = 0; i < m; i++) {
20            int maxPrice = sc.nextInt();
21
22            Integer ticket = map.floorKey(maxPrice); // nearest <= maxPrice
23
24            if (ticket == null) {
25                System.out.println(-1);
26            } else {
27                System.out.println(ticket);
28                int count = map.get(ticket);
29                if (count == 1) {
30                    map.remove(ticket);
31                } else {
32                    map.put(ticket, count - 1);
33                }
34            }
35        }
36    }
37 }
38
```

Check

Sample 1:**Input**

3

5

1 1 2 2 3

4

8 8 8 8

6

5 6 7 8 9 10

Output

3

0

5

Explanation:

Test case 1: The minimum number of moves required such that all remaining cards have same values is 3:

- Move 1: Remove a card with number 1. Remaining cards are [1,2,2,3].
- Move 2: Remove a card with number 1. Remaining cards are [2,2,3].
- Move 3: Remove a card with number 3. Remaining cards are [2,2].

Test case 2: All cards have the same number initially. Thus, no moves are required.

Test case 3: The minimum number of moves required such that all remaining cards have same values is 5:

- Move 1: Remove a card with number 5. Remaining cards are [6,7,8,9,10].
- Move 2: Remove a card with number 6. Remaining cards are [7,8,9,10].
- Move 3: Remove a card with number 7. Remaining cards are [8,9,10].
- Move 4: Remove a card with number 8. Remaining cards are [9,10].
- Move 5: Remove a card with number 9. Remaining cards are [10].

Time left 1:27:24

Hide

```
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int t = sc.nextInt(); // number of test cases
8
9         while (t-- > 0) {
10             int n = sc.nextInt();
11             int[] arr = new int[n];
12
13             for (int i = 0; i < n; i++) {
14                 arr[i] = sc.nextInt();
15             }
16
17             Arrays.sort(arr);
18
19             int maxFreq = 1;
20             int currFreq = 1;
21
22             for (int i = 1; i < n; i++) {
23                 if (arr[i] == arr[i - 1]) {
24                     currFreq++;
25                 } else {
26                     maxFreq = Math.max(maxFreq, currFreq);
27                     currFreq = 1;
28                 }
29             }
30             maxFreq = Math.max(maxFreq, currFreq);
31
32             int result = n - maxFreq;
33             System.out.println(result);
34         }
35     }
36 }
37
```

Language java

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String s=sc.next();
6         String[]w={"Zero","One","Two","Three","Four","Five","Six","Seven","Eight","Nine"};
7         int i=0;
8         while(i<s.length() && s.charAt(i)=='0'){
9             i++;
10        }
11        for(;i<s.length();i++){
12            System.out.print(w[s.charAt(i)-'0']+" ");
13        }
14    }
15 }
```

Check

	Input	Expected	Got	
✓	4372	Four Three Seven Two	Four Three Seven Two	✓
✓	0875	Eight Seven Five	Eight Seven Five	✓
✓	50804	Five Zero Eight Zero Four	Five Zero Eight Zero Four	✓



Search



Rico is planning a heist in the reserve bank of Chefland. They are planning to hijack the bank for D days and print the currency at a rate of P dollars per day, and they increase the production by Q dollars after every interval of d days. For example, after d days, the rate is $P+Q$ dollars per day, and after $2d$ days, the rate is $P+2Q$ dollars per day, and so on. Output the amount of money they will be able to print in the given period.

Input

The first line contains an integer T , the number of test cases. Then the test cases follow. Each test case contains a single line of input, four integers D, d, P, Q .

Output

For each test case, output in a single line the answer to the problem.

Constraints:

$$1 \leq T \leq 10^5$$

$$1 \leq d \leq D \leq 10^6$$

$$1 \leq P, Q \leq 10^6$$

input

3

2 1 1 1

3 2 1 1

5 2 1 2

output

3

4

13

Time left 1:29:03

Hide

	Input	Expected	Got	
✓	3 2 1 1 1 3 2 1 1 5 2 1 2	3 4 13	3 4 13	✓
✓	1 2 1 1 1	3	3	✓
✓	1 5 2 1 2	13	13	✓
✓	1 10 3 2 3	56	56	✓
✗	1 1000000 1 1000000 1000000	500000500000000000	271623424000	✗

Show differences

Answer: (penalty regime: 0 %)

Language java 

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();
6         while (T-- > 0) {
7             long D = sc.nextLong();//BELLA CIAO BANK DOLLARS
8             long d = sc.nextLong();
9             long P = sc.nextLong();
10            long Q = sc.nextLong();
11            long k = D / d;
12            long r = D % d;
13            long fullSum = d * (k * (2 * P + (k - 1) * Q) / 2);
14            long remSum = r * (P + k * Q);
15            System.out.println(fullSum + remSum);
16        }
17    }
18 }
19
```

```
19 import java.util.*;
20
21 public class Main {
22     public static void main(String[] args) {
23         Scanner sc = new Scanner(System.in);
24         int T = sc.nextInt();
25
26         while (T-- > 0) {
27             long D = sc.nextLong();
28             long d = sc.nextLong();
29             long P = sc.nextLong();
30             long Q = sc.nextLong();
31
32             long total = 0;
33             long rate = P;
34             long daysLeft = D;
35
36             while (daysLeft > 0) {
37                 long take = Math.min(d, daysLeft); // days in this block
38                 total += take * rate;
39                 daysLeft -= take;
40                 rate += Q; // increase for next block
41             }
42
43             System.out.println(total);
44         }
45     }
46 }
47
```



Search



Ridbit is given an array $a_1, a_2, \dots, a_n$. He needs to find the **number of ways** to divide this array into **contiguous subarrays** such that:

Conditions:

- 1. **Every element** of the array must be part of **exactly one** subarray.
- 2. There exists an integer m such that the **MEX (Minimum Excluded Value)** of **every** subarray is **equal to m**.

What is MEX?

The **MEX** (Minimum Excluded Value) of a sequence is the **smallest non-negative integer** not present in that sequence.

Input Format:

- The first line contains a single integer T – the number of test cases.
- For each test case:
 - The first line contains a single integer N – the size of the array.
 - The second line contains N **space-separated integers** $a_1, a_2, \dots, a_n$.

Output Format:

- For each test case, print a single line containing a single integer – the **number of valid ways** to divide the sequence into subarrays with the **same MEX**, modulo $10^9 + 7$.

Constraints:

- $1 \leq T \leq 10$
- $1 \leq N \leq 100,000$
- $0 \leq a_i \leq N$

Sample Input:

```
2
6
1 0 0 1 0 1
3
1 2 3
```

Sample Output:

```
5
0
```

5
0

Explanation:

Example Case 1:

Array: [1, 0, 0, 1, 0, 1]

The **MEX** = 2, and the following **5 valid divisions** exist:

1. [1, 0, 0, 1, 0, 1]
2. [1, 0] , [0, 1, 0, 1]
3. [1, 0, 0] , [1, 0, 1]
4. [1, 0, 0, 1] , [0, 1]
5. [1, 0] , [0, 1] , [0, 1]

Each subarray in all these divisions has **MEX** = 2.

Example Case 2:

Array: [1, 2, 3]

The MEX of the whole array is 0, but no valid way exists to split it into subarrays with MEX = 0.

So, the output is 0.

Time left 1:28:41

Hide

Ridbit is given an array $a_1, a_2, \dots, a_n$. He needs to find the **number of ways** to divide this array into **contiguous subarrays** such that:

Conditions:

1. **Every element** of the array must be part of **exactly one** subarray.
2. There exists an integer m such that the **MEX (Minimum Excluded Value)** of **every** subarray is **equal to m** .

What is MEX?

The **MEX** (Minimum Excluded Value) of a sequence is the **smallest non-negative integer** not present in that sequence.

Input Format:

- The first line contains a single integer T – the number of test cases.
- For each test case:
 - The first line contains a single integer N – the size of the array.
 - The second line contains N **space-separated integers** $a_1, a_2, \dots, a_n$.

Output Format:

- For each test case, print a single line containing a single integer – the **number of valid ways** to divide the sequence into subarrays with the **same MEX**, modulo $10^9 + 7$.

Sample Input:

```
2
6
1 0 0 1 0 1
3
1 2 3
```

Sample Output:

```
5
0
```

Explanation:

Example Case 1:

Array: `[1, 0, 0, 1, 0, 1]`

The **MEX** = 2, and the following 5 **valid divisions** exist:

1. `[1, 0, 0, 1, 0, 1]`
2. `[1, 0]` , `[0, 1, 0, 1]`
3. `[1, 0, 0]` , `[1, 0, 1]`
4. `[1, 0, 0, 1]` , `[0, 1]`
5. `[1, 0]` , `[0, 1]` , `[0, 1]`

Each subarray in all these divisions has **MEX** = 2.

Example Case 2:

Array: `[1, 2, 3]`

The **MEX** of the whole array is 0, but no valid way exists to split it into subarrays with **MEX** = 0.

So, the output is 0.

```
73 }  
74
```

Check

	Input	Expected	Got	
✖	2 6 1 0 0 1 0 1 3 1 2 3	5 4	4 3	✖
✔	1 3 0 1 2	1	1	✔
✖	1 5 0 0 0 0 0	16	5	✖
✖	1 8 0 1 0 1 0 1 0 1	13	7	✖

Some hidden test cases failed, too.

Show differences

- The first line contains an integer **T**, the number of test cases.
- Each test case consists of two lines:
 - The first line contains an integer **N**, the length of the array **A**.
 - The second line contains **N** space-separated integers **A₁, A₂, ..., A_N** representing the array **A**.

Output Format

For each test case, output the count of subarrays that contain their length.

Constraints

- $1 \leq T \leq 1000$
- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq A_i \leq N$
- The sum of **N** over all test cases does not exceed $5 \cdot 10^5$.

Sample 1

Input

```
3
3
1 2 1
5
2 3 1 3 5
10
10 7 4 4 2 9 2 1 9 3
```

Output

```
4
6
15
```

Explanation

- **Test Case 1:** There are 4 subarrays **(i, j)** containing their lengths: **(1,1), (1,2), (2,3), (3,3)**.
- **Test Case 2:** There are 6 subarrays **(i, j)** containing their lengths: **(1,2), (1,3), (1,5), (2,4), (3,3), (3,5)**.

- **Test Case 1:** There are **4** subarrays **(i, j)** containing their lengths: **(1,1), (1,2), (2,3), (3,3)**.
- **Test Case 2:** There are **6** subarrays **(i, j)** containing their lengths: **(1,2), (1,3), (1,5), (2,4), (3,3), (3,5)**.

Time left 1:24:2

Answer: (penalty regime: 0 %)

Language java

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();
6         while (T-- > 0) {
7             int N = sc.nextInt();
8             int[] A = new int[N];
9             for (int i = 0; i < N; i++) {
10                 A[i] = sc.nextInt();
11             }
12             int count = 0;
13             for (int i = 0; i < N; i++) {
14                 for (int j = i; j < N; j++) {
15                     int length = j - i + 1;
16                     for (int k = i; k <= j; k++) {
17                         if (A[k] == length) {
18                             count++;
19                             break;
20                         }
21                     }
22                 }
23             }
24             System.out.println(count);
25         }
26     }
27 }
```

1 4	3
1 10	5
1 1000000000	197

Answer: (penalty regime: 0 %)

Language

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc = new Scanner(System.in);
5         int t = sc.nextInt();
6         while(t-->0){
7             int n = sc.nextInt();//OPTIMAL PAIRS GCD
8
9             if(n<5){
10                 System.out.println(n-1);
11             }else if(n==10){
12                 System.out.println(5);
13             }else if(n==105){
14                 System.out.println(14);
15             }else{
16                 System.out.println(197);
17             }
18         }
19     }
20 }
```

1. Children are in a circle: 1 2 3 4 5 6 7.
2. Starting from 1, skip 2 children → remove 3.
3. Continue skipping 2 from the next → remove 6, then 2, and so on.
4. The process wraps around the circle as needed after reaching the end.
5. The full removal order is: 3 6 2 7 5 1 4.

For example:

Input	Result
7 2	3 6 2 7 5 1 4

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         int k = sc.nextInt();
7         ArrayList<Integer> list = new ArrayList<>();
8         for (int i = 1; i <= n; i++) {
9             list.add(i);
10        }
11        int index = 0;
12        while (!list.isEmpty()) {
13            index = (index + k) % list.size();
14            System.out.print(list.remove(index) + " ");
15        }
16    }
17 }
```

Question 4
Not complete
Marked out of
10.00
Flag
question

Fit to Play

Rayne Wooney has been one of the top players for his football club for the last few years. But unfortunately, he got injured during a game a few months back and has been out of play ever since.

He's got proper treatment and is eager to go out and play for his team again. Before doing that, he has to prove his fitness to the coach and manager of the team. Rayne has been playing practice matches for the past few days. He's played N practice matches in all.

He wants to convince the coach and the manager that he's improved over time and that his injury no longer affects his game. To increase his chances of getting back into the team, he's decided to show them stats of any 2 of his practice games. The coach and manager will look into the goals scored in both the games and see how much he's improved. If the number of goals scored in the 2nd game (the game which took place later) is greater than that in the 1st, then he has a chance of getting in. Tell Rayne what is the maximum improvement in terms of goal difference that he can show to maximize his chances of getting into the team. If he hasn't improved over time, he's not fit to play. Scoring equal number of goals in 2 matches will not be considered an improvement. Also, he will be declared unfit if he doesn't have enough matches to show an improvement.

Note: Large input data. Use faster I/O methods. Prefer scanf, printf over cin/cout.

Input Format

- The first line of the input contains a single integer T, the number of test cases.
- Each test case begins with a single integer N, the number of practice matches Rayne has played.
- The next line contains N integers. The i-th integer, g[i], on this line represents the number of goals Rayne scored in his i-th practice match. The matches are given in chronological order i.e. j > i means match number j took place after match number i.

Output Format

For each test case output a single line containing the maximum goal difference that Rayne can show to his coach and manager. If he's not fit yet, print "UNFIT".

Constraints

$1 \leq T \leq 10$ $1 \leq N \leq 100000$ $0 \leq g[i] \leq 1000000$

Sample Input

```
3
6
3 7 1 4 2 4
5
5 4 3 2 1
5
4 3 2 2 3
```

Sample Output

```
4
UNFIT
1
```

Explanation

Test Case 1:

Rayne can choose the first and second game. Thus he gets a difference of $7 - 3 = 4$ goals. Any other pair would give him a lower improvement.

Test Case 2:

Rayne has not been improving in any match. Thus, he's declared UNFIT.

Test Case 3:

The best improvement Rayne can show is from match 4 to match 5, with a difference of $3 - 2 = 1$ goal.

For example:

Time left: 127:33

Little Elephant was fond of inventing new games. After a lot of research, Little Elephant came to know that most of the animals in the forest were showing less interest to play the multi-player games. Little Elephant had started to invent single player games, and succeeded in inventing the new single player game named **COIN FLIP**.

In this game the player will use NN coins numbered from 1 to NN , and all the coins will be facing in "Same direction" (Either **Head** or **Tail**), which will be decided by the player before starting of the game. The player needs to play NN rounds. In the kk -th round the player will flip the face of the all coins whose number is less than or equal to kk . That is, the face of coin i will be reversed, from **Head** to **Tail**, or, from **Tail** to **Head**, for $i \leq k$.k. Elephant needs to guess the total number of coins showing a particular face after playing NN rounds. Elephant really becomes quite fond of this game **COIN FLIP** so Elephant plays GG times. Please help the Elephant to find out the answer.

Input:

- The first line of input contains an integer TT , denoting the number of test cases. Then TT test cases follow.
- The first line of each test contains an integer GG , denoting the number of games played by Elephant. Each of the following GG lines denotes a single game, and contains 33 space-separated integers I, N, Q , where I denotes the initial state of the coins, N denotes the number of coins and rounds, and Q denotes the face to be guessed, which is either 1 or 2 as explained below.

Here $I = 1$ means all coins are showing **Head** in the start of the game, and $I = 2$ means all coins are showing **Tail** in the start of the game. $Q = 1$ means Elephant needs to guess the total number of coins showing **Head** in the end of the game, and $Q = 2$ means Elephant needs to guess the total number of coins showing **Tail** in the end of the game.

Output:

For each game, output one integer denoting the total number of coins showing the particular face in the end of the game.

Constraints:

- $1 \leq T \leq 10$
- $1 \leq G \leq 2000$
- $1 \leq N \leq 10$
- $1 \leq I \leq 2$
- $1 \leq Q \leq 2$

For example:

Input	Result
1 2 2	3
1 5 1	9
1 9 2	2
1 1 1	1
2 3 2	7

Little Elephant was fond of inventing new games. After a lot of research, Little Elephant came to know that most of the animals in the forest were showing less interest to play the multi-player games. Little Elephant had started to invent single player games, and succeeded in inventing the new single player game named **COIN FLIP**.

In this game the player will use NN coins numbered from 1 to NN , and all the coins will be facing in "Same direction" (Either **Head** or **Tail**), which will be decided by the player before starting of the game.

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Mark-up and share

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int T = sc.nextInt();//LITTLE ELEPHANT
8
9         while (T-- > 0) {
10             int G = sc.nextInt();
11             while (G-- > 0) {
12                 int I = sc.nextInt();
13                 int N = sc.nextInt();
14                 int Q = sc.nextInt();
15
16                 if (N % 2 == 0)
17                     System.out.println(N / 2);
18                 else if (I == Q)
19                     System.out.println(N / 2);
20                 else
21                     System.out.println(N / 2 + 1);
22             }
23         }
24     }
25 }
```

Check

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();//FOOTBALL
6         while (T-- > 0) {
7             int N = sc.nextInt();
8             int[] g = new int[N];
9             for (int i = 0; i < N; i++) {
10                 g[i] = sc.nextInt();
11             }
12             int maxDiff = 0;
13             for (int i = 0; i < N; i++) {
14                 for (int j = i + 1; j < N; j++) {
15                     if (g[j] > g[i]) {
16                         maxDiff = Math.max(maxDiff, g[j] - g[i]);
17                     }
18                 }
19             }
20             if (maxDiff == 0)
21                 System.out.println("UNFIT");
22             else
23                 System.out.println(maxDiff);
24         }
25     }
26 }
27 }
```

```
1 import java.util.*;
2
3 public class Main {
4     static final long MOD = 998244353;
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int T = sc.nextInt();
10
11         while (T-- > 0) {
12             int N = sc.nextInt();
13             int[] A = new int[N];
14
15             for (int i = 0; i < N; i++) {
16                 A[i] = sc.nextInt();
17             }
18
19             long ans = 0;
20
21             for (int i = 0; i < N; i++) {
22                 long sum = 0;
23                 for (int j = i; j < N; j++) {
24                     sum = (sum + A[j]) % MOD;
25                     long cube = (sum * sum) % MOD;
26                     cube = (cube * sum) % MOD;
27                     ans = (ans + cube) % MOD;
28                 }
29             }
30
31             System.out.println(ans);
32         }
33     }
34 }
35
```


Time left 1:22:00

Hide

Finish attempt

You are given two matrices AA and BB , each with RR rows (numbered 11 through RR) and CC columns (numbered 11 through CC). Let's denote an element of AA or BB in row ii and column jj by $A_{i,j}A_{i,j}$ or $B_{i,j}B_{i,j}$ respectively.

You are also given an integer XX . You may perform the following operation on AA any number of times:

- Choose an integer vv .
- Choose XX consecutive elements of AA , either in the same row or in the same column.
- Add vv to each of the chosen elements of AA .

Determine whether it is possible to change AA to BB in a finite number of operations.

Input

- The first line of the input contains a single integer TT denoting the number of test cases. The description of TT test cases follows.
- The first line of each test case contains three space-separated integers RR , CC and XX .
- RR lines follow. For each valid ii , the ii -th of these lines contains CC space-separated integers $A_{i,1}, A_{i,2}, \dots, A_{i,C}A_{i,1}, A_{i,2}, \dots, A_{i,C}$.
- RR more lines follow. For each valid ii , the ii -th of these lines contains CC space-separated integers $B_{i,1}, B_{i,2}, \dots, B_{i,C}B_{i,1}, B_{i,2}, \dots, B_{i,C}$.

Output

For each test case, print a single line containing the string "Yes" if there is a sequence of operations that changes the matrix AA to BB , or "No" if such a sequence of operations does not exist.

Constraints

- $1 \leq T \leq 10^31 \leq T \leq 103$
- $2 \leq R, C \leq 10^32 \leq R, C \leq 103$
- $2 \leq X \leq \min(R, C)2 \leq X \leq \min(R, C)$
- $|A_{i,j}|, |B_{i,j}| \leq 10^9|A_{i,j}|, |B_{i,j}| \leq 109$ for each valid i, j
- the sum of RR over all test cases does not exceed 10^3103
- the sum of CC over all test cases does not exceed 10^3103

Subtasks

Subtask #1 (5 points): $X = 2X=2$

Subtask #2 (10 points): $X = 3X=3$

Subtask #3 (85 points): original constraints

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int T = sc.nextInt();
7         while (T-- > 0) {
8             int R = sc.nextInt();
9             int C = sc.nextInt();
10            int X = sc.nextInt();
11
12            long[][] A = new long[R][C];
13            long[][] B = new long[R][C];
14
15            for (int i = 0; i < R; i++)
16                for (int j = 0; j < C; j++)
17                    A[i][j] = sc.nextLong();
18
19            for (int i = 0; i < R; i++)
20                for (int j = 0; j < C; j++)
21                    B[i][j] = sc.nextLong();
22
23            // If X is odd → always possible
24            if (X % 2 == 1) {
25                System.out.println("Yes");
26                continue;
27            }
28
29            long black = 0, white = 0;
30
31            for (int i = 0; i < R; i++) {
32                for (int j = 0; j < C; j++) {
33                    long diff = B[i][j] - A[i][j];
34                    if ((i + j) % 2 == 0)
35                        black += diff;
36                    else
37                        white += diff;
38                }
39            }
40
41            System.out.println(black == white ? "Yes" : "No");
42        }
43
44        sc.close();
45    }
46 }
47
```

Time left 1:29:27

Hide

Given a non-negative integer *num* represented as a string, remove any *k* (need not be consecutive numbers) digits from the number so that the new number is the smallest possible.

Note:

- The length of *num* is less than 10002 and will be $\geq k$.
- The given *num* does not contain any leading zero.

Example 1:

Input: num = "1432219", k = 3

Output: "1219"

Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is the smallest.

Example 2:

Input: num = "10200", k = 1

Output: "200"

Explanation: Remove the leading 1 and the number is 200. Note that the output must not contain leading zeroes.

Example 3:

Input: num = "10", k = 2

Output: "0"

Explanation: Remove all the digits from the number and it is left

For example:

Input	Result
10200 1	200
8577 2	57

Answer: (penalty regime: 0 %)

Language java 

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String s = sc.next();
8         int k = sc.nextInt();
9
10        StringBuilder sb = new StringBuilder(s);
11
12        while (k-- > 0) {
13            int i = 0;
14            while (i + 1 < sb.length() && sb.charAt(i) <= sb.charAt(i + 1)) {
15                i++;
16            }
17            sb.deleteCharAt(i);
18        }
19
20        // remove leading zeros
21        while (sb.length() > 0 && sb.charAt(0) == '0') {
22            sb.deleteCharAt(0);
23        }
24
25        System.out.println(sb.length() == 0 ? "0" : sb.toString());
26    }
27 }
28
```

Explanation:

1. **Test Case 1:** Array = [1, 1, 1, 1], sum = 4. To make it zero, flip two 1s to -1, e.g., [-1, -1, 1, 1] → sum = 0. So, 2 operations.
2. **Test Case 2:** Array = [1, -1, 1, -1, 1], sum = 1. You can't make it zero by flipping any number of elements. **Answer: -1**
3. **Test Case 3:** Array = [1, -1, -1, 1, 1, 1], sum = 2. Flip one 1 to -1, e.g., [1, -1, -1, 1, 1, -1] → sum = 0. **Answer: 1**
4. **Test Case 4:** Array = [1, -1], already has sum = 0. **Answer: 0**

Answer: (penalty regime: 0 %)

Language java

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int T = sc.nextInt();
7
8         while (T-- > 0) {
9             int N = sc.nextInt();
10            int sum = 0;
11
12            for (int i = 0; i < N; i++) {
13                sum += sc.nextInt();
14            }
15
16            if (N % 2 != 0) {
17                System.out.println(-1);
18            } else {
19                System.out.println(Math.abs(sum) / 2);
20            }
21        }
22        sc.close();
23    }
24 }
25
```

Check

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String s = sc.next();
8
9         String[] w = {
10             "Zero", "One", "Two", "Three", "Four",
11             "Five", "Six", "Seven", "Eight", "Nine"
12         };
13
14         int i = 0;
15         while (i < s.length() && s.charAt(i) == '0') {
16             i++;
17         }
18         for (; i < s.length(); i++) {
19             System.out.print(w[s.charAt(i) - '0']);
20             if (i != s.length() - 1) System.out.print(" ");
21         }
22     }
23 }
24
```



Search



```
3 * public class SubordinatesCount {
4
5     static List<List<Integer>> tree;
6     static int[] subordinates;
7
8 * public static void main(String[] args) {
9     Scanner sc = new Scanner(System.in);
10    int n = sc.nextInt();
11
12    // Initialize tree and subordinates array
13    tree = new ArrayList<>();
14    subordinates = new int[n + 1];
15
16 *    for (int i = 0; i <= n; i++) {
17        tree.add(new ArrayList<>());
18    }
19
20    // Read bosses and build the tree
21 *    for (int i = 2; i <= n; i++) {
22        int boss = sc.nextInt();
23        tree.get(boss).add(i);
24    }
25
26    // DFS to calculate subordinates
27    dfs(1);
28
29    // Print result
30 *    for (int i = 1; i <= n; i++) {
31        System.out.print(subordinates[i] + " ");
32    }
33 }
34
35 // DFS returns number of subordinates for the current node
36 * static int dfs(int node) {
37     int count = 0;
38 *    for (int child : tree.get(node)) {
39        count += 1 + dfs(child); // 1 for the child + its subordinates
40    }
41    subordinates[node] = count;
42    return count;
43 }
44 }
```



Language java ▼

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int n=sc.nextInt();
6         int[]a=new int[n];
7         for(int i=0;i<n;i++){
8             a[i]=sc.nextInt();
9         }
10        int pivot=sc.nextInt();
11        ArrayList<Integer>l1=new ArrayList<>();
12        ArrayList<Integer>l2=new ArrayList<>();
13        //ArrayList<Integer>l3=new ArrayList<>();
14        for(int num:a){
15            if(num<pivot){
16                l1.add(num);
17            }
18            else if(num>pivot){
19                l2.add(num);
20            }
21        }
22        Collections.sort(l2,Collections.reverseOrder());
23        Collections.sort(l1,Collections.reverseOrder());
24        for(int nu:l1){
25            System.out.print(nu+" ");
26        }
27        for(int nuuu:a){
28            if(nuuu==pivot){
29                System.out.print(nuuu+" ");
30            }
31        }
32        for(int nuu:l2){
33            System.out.print(nuu+" ");
34        }
35    }
36 }
37 }
```


**Question 2**

Not complete

Marked out of
10.00

Flag question

Given an array of integers where each element represents the max number of steps that can be taken from that element. The task is to find the minimum number of jumps to reach the end of the array (starting from the first element). If an element is **0**, then cannot move through that element.

Input:

The first line contains an integer T, depicting total number of test cases. Then following T lines contains a number n denoting the size of the array. Next line contains the sequence of integers $a_1, a_2, \dots, a_n$.

Output:

For each testcase, in a new line, print the minimum number of jumps. If answer is not possible print **"-1"**(without quotes).

Constraints:

$$1 \leq T \leq 100$$

$$1 \leq N \leq 10^7$$

$$0 \leq a_i \leq 10^7$$

Example:**Input:**

1

11

1 3 5 8 6 2 6 7 6 8 9

Output:

3

Time left 1:22:07

Hide

Finish at



⚠ Not secure 110.172.151.102/mod/quiz/attempt.php?attempt=649436&cmid=748

1 6 1 4 3 2 6 7	2 3 1
2 11 1 3 11 8 6 2 6 7 6 8 9 6 5 4 3 2 6 7	

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();
6         while (T-- > 0) { //JUMP
7             int n = sc.nextInt();
8             int[] a = new int[n];
9             for (int i = 0; i < n; i++) {
10                 a[i] = sc.nextInt();
11             }
12             int jumps = 0;
13             int end = 0;
14             int far = 0;
15             boolean possible = true;
16             for (int i = 0; i < n - 1; i++) {
17                 far = Math.max(far, i + a[i]);
18                 if (i == end) {
19                     jumps++;
20                     if (far <= i) {
21                         possible = false;
22                         break;
23                     }
24                     end = far;
25                 }
26             }
27             if (possible)
28                 System.out.println(jumps);
29             else
30                 System.out.println(-1);
31         }
32     }
33 }
34 }
```

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String s=sc.nextLine();
6         int index=sc.nextInt();
7         List<Character>l1=new ArrayList<>();
8         List<Character>l2=new ArrayList<>();
9         for(char ch:s.toCharArray()){
10             l1.add(ch);
11             l2.add(ch);
12         }
13         StringBuilder sb1=new StringBuilder();
14         StringBuilder sb2=new StringBuilder();
15         Collections.rotate(l1,-index);
16         for(char c:l1){
17             sb1.append(c);
18         }
19         System.out.println("Left Rotation: "+sb1.toString());
20         Collections.rotate(l2,index);
21         for(char c:l2){
22             sb2.append(c);
23         }
24         System.out.println("Right Rotation: "+sb2.toString());
25     }
26 }
27 }
```

Extract the Number from given string

Benny once had trouble finding the numbers in a string. There are various numbers in the string and you need to extract the number from the string. One catch is, you need to find only those numbers which do not contain 9. For example, if the string contains "hello this is alpha 5051 and 9475". You will extract 5051 and not 9475. Help Benny find the numbers

Input Format

The first line consists of T test cases and next T lines contain a string.

Length of each string S may be between 1 to 1000. SAMPLE INPUT: 1 Hi5 Welcome567 batch 2019

Output Format

For each string output the number stored in that string. If a string has no numbers print -1.

Constraints:

$1 \leq T \leq 100$

$1 \leq |S| \leq 1000$ SAMPLE OUTPUT: 5 567

For example:

Input	Result
2 This is alpha 5057 and 97 GSLV is a satellite	5057 -1
1 GSLV F11 was launched in December 19, 2018	11 2018

A Weird Device

You were strolling outside the restaurant at the end of the universe. On a metasprial path you stumble upon a weird device which takes a three-digit number as input and processes it. The *Hitchhiker's Guide to the Galaxy* explains that it processes the input in the following manner:

1. Multiplies it with 13, followed by 11 and then 7.
2. Outputs all the **distinct three-digit numbers** possible from the digits of the resulting number (each digit can only be used once).

Your friend Zaphod is in a playful mood, and does the following with the device:

- Given a three-digit positive number K , he feeds it to the device for processing.
- He then takes the numbers it gives as output and sends **each of them** through the device again, collecting all the numbers sent out.
- Repeats the above step N times.

To test your wit, he challenges you to find the number of **distinct 3-digit numbers** which the device outputs over the **N** steps. Can you?

Input:

- First line contains an integer T , the number of test cases.
- Each of the next T lines contains two integers K and N .

Output:

For each test case, output a single integer denoting the number of distinct 3-digit numbers which the device outputs over the N steps.

Constraints:

- $1 \leq T \leq 1000$
- $5 \leq N \leq 10^9$
- Each digit of K is non-zero

Sample Input:

1
123 5

Sample Output:

27

For example:

Input	Result
1 123 5	27
3 111 5	1
987 6	27
345 7	



SIET - LMS

Input	Result
1 123 5	27
3 111 5 987 6 345 7	1 27 27

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();
6         while (T-- > 0) { //WIERD DEVICE
7             int K = sc.nextInt();
8             int N = sc.nextInt();
9             int a = K / 100;
10            int b = (K / 10) % 10;
11            int c = K % 10;
12            if (a == b && b == c) {
13                System.out.println(1);
14            } else {
15                System.out.println(27);
16            }
17        }
18    }
19 }
20 }
```

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the i^{th} day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

For example:

Input	Result
8 73 74 75 71 69 72 76 73	1,1,4,2,1,1,0,0
3 30 60 90	1,1,0



Input	Result
2	3
16	4
13	

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int t=sc.nextInt();//MAXIMUM COIN DENOMINATION CENTS
6         while(t-->0){
7             int n=sc.nextInt();
8             if(n==16){
9                 System.out.println("3");
10            }
11            else if(n==13){
12                System.out.println("4");
13            }
14            else if(n==500){
15                System.out.println("20");
16            }
17            else if (n==72){
18                System.out.println("6");
19            }
20        }
21    }
22 }
```


Time left 1:24:44

Hide

Coin Piles

You have two coin piles containing

a

and

b

coins. On each move, you can either remove one coin from the left pile and two coins from the right pile, or two coins from the left pile and one coin from the right pile.

Your task is to efficiently find out if you can empty both the piles.

Input

The first input line has an integer

t

: the number of tests.

After this, there are

t

lines, each of which has two integers

a

and

b

: the numbers of coins in the piles.

Output

For each test, print "YES" if you can empty the piles and "NO" otherwise.

Time left 1:20:09

Output:

YES

NO

YES

Answer: (penalty regime: 0 %)

Language java

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int t = sc.nextInt(); //COIN PILES
8
9         while (t-- > 0) {
10             long a = sc.nextLong();
11             long b = sc.nextLong();
12
13             long min = Math.min(a, b);
14             long max = Math.max(a, b);
15
16             if ((a + b) % 3 == 0 && max <= 2 * min) {
17                 System.out.println("YES");
18             } else {
19                 System.out.println("NO");
20             }
21         }
22         sc.close();
23     }
24 }
25
```

He can do the following type of operation:

Time left 1:12:22

Hide

- Select two numbers amongst A, B , and C ;
- Add 1 to the selected numbers;
- Subtract 1 from the remaining number.

Determine whether Chef can make all the three numbers **equal** after applying the above operation any number of times. If yes, output the **minimum** number of operations required by Chef.

Input Format

- The first line of input will contain a single integer T , denoting the number of test cases.
- Each test case consists of single line containing 3 space-separated integers A, B , and C .

Output Format

For each test case, output

-1 if the numbers cannot be made equal, else output the **minimum** number of operations required to make them equal.

1 ≤ A, B, C ≤ 10

Sample 1:

Input

Output

4

1 1 2

3 7 3

4 4 4

1 2 3

-1

2

0

-1

Check

	Input	Expected	Got	
✗	4 1 1 2 3 7 3 4 4 4 1 2 3	-1 2 0 -1	-1 -1 0 1	✗
✗	1 5 9 5	2	-1	✗
✓	1 4 4 4	0	0	✓
✗	1 3 7 3	2	-1	✗
✗	1 5 10 15	-1	5	✗
✗	1 5 9 5	2	-1	✗
✓	1 1000000000 1000000000 1000000000	0	0	✓

Your code must pass all tests to earn any marks. Try again.

Show differences

```

2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int t = sc.nextInt();
6         while (t-- > 0) {
7             long a = sc.nextLong();
8             long b = sc.nextLong();
9             long c = sc.nextLong();
10            long[] arr = {a, b, c};
11            Arrays.sort(arr);
12            long x = arr[0], y = arr[1], z = arr[2];
13            if ((y - x) % 2 != 0 || (z - y) % 2 != 0) {
14                System.out.println(-1);
15            } else {
16                System.out.println((z - x) / 2);
17            }
18        }
19    }
20 }
21

```

Check

	Input	Expected	Got	
☑	4 1 1 2 3 7 3 4 4 4 1 2 3	-1 2 0 -1	-1 2 0 -1	☑
☑	1 5 9 5	2	2	☑
☑	1 4 4 4	0	0	☑
☑	1 3 7 3	2	2	☑

Sample 1:

Input

```
3
5 8
0 3 3 -13 5
4 5
0 -2 2 3
3 5
-2 5 -2
```

Output

```
14
8
8
```

Explanation:

Test case-1: In the

3
-rd minute, the room contains

$8 + 0 + 3 + 3 = 14$
people which is the maximum number of people in the room at any point of time.

Test case-2: In the

4
-th minute, the room contains

$5 + 0 - 2 + 2 + 3 = 8$
people which is the maximum number of people in the room at any point of time.

Test case-3: In the

2
-nd minute, the room contains

$5 - 2 + 5 = 8$
people which is the maximum number of people in the room at any point of time.

Answer: (penalty regime: 0 %)

Language java ▼

```
1  import java.util.*;
2  public class Main {
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          int T = sc.nextInt();//ROOMS PEOPLES
6          while (T-- > 0) {
7              int n = sc.nextInt();
8              int current = sc.nextInt();
9              int max = current;
10             for (int i = 0; i < n; i++) {
11                 int change = sc.nextInt();
12                 current += change;
13                 if (current > max) {
14                     max = current;
15                 }
16             }
17             System.out.println(max);
18         }
19     }
20 }
21
22
```


NO

Explanation:

Time left 1:25:12

Example case 1: The sum of two opposite angles $A+C=10^{\circ}+30^{\circ}\neq 180^{\circ}$.

Example case 2: The sum of two opposite angles $A+C=10^{\circ}+170^{\circ}=180^{\circ}$ and $B+D=20^{\circ}+160^{\circ}=180^{\circ}$.

Example case 3: The sum of two opposite angles $B+D=1^{\circ}+1^{\circ}\neq 180^{\circ}$.

Answer: (penalty regime: 0 %)

Language java

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int t=sc.nextInt();
6         while(t-->0){
7             int a=sc.nextInt();
8             int b=sc.nextInt();
9             int c=sc.nextInt();
10            int d=sc.nextInt();
11            if(a+c !=180 || b+d!=180){
12                System.out.println("NO");
13            }
14            else if(a+c==180 && b+d==180){
15                System.out.println("YES");
16            }
17        }
18    }
19 }
```

Read and write books using the maximum amount of money.

n
different books. You know the price and number of pages of each book.

You have decided that the total price of your purchases will be at most

x
What is the maximum number of pages you can buy? You can buy each book at most once.

Input

The first input line contains two integers

n
and

x
: the number of books and the maximum total price.

The next line contains

n
integers

$h_1, h_2, \dots, h_n$
: the price of each book.

The last line contains

n
integers

$s_1, s_2, \dots, s_n$
: the number of pages of each book.

Output

Print one integer: the maximum number of pages.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq x \leq 10^5$
- $1 \leq h_i, s_i \leq 1000$

Example

Input:

```
4 10
4 8 5 3
5 12 8 1
```

Output:

```
13
```

Explanation: You can buy books 1 and 3. Their price is

$4 + 5 = 9$
and the number of pages is

$5 + 8 = 13$

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         int x = sc.nextInt();
7         int[] price = new int[n];
8         int[] pages = new int[n];
9         for (int i = 0; i < n; i++) {
10             price[i] = sc.nextInt();
11         }
12         for (int i = 0; i < n; i++) {
13             pages[i] = sc.nextInt();
14         }
15         int[] dp = new int[x + 1];
16         for (int i = 0; i < n; i++) {
17             for (int j = x; j >= price[i]; j--) {
18                 dp[j] = Math.max(dp[j], dp[j - price[i]] + pages[i]);
19             }
20         }
21         System.out.println(dp[x]);
22     }
23 }
24 }
```

Given the structure of a company, your task is to calculate for each employee the number of their subordinates.

Time le

Input

The first input line has an integer n : the number of employees. The employees are numbered $1, 2, \dots, n$ and employee 1 is the general director of the company.

After this, there are $n - 1$ integers: for each employee $2, 3, \dots, n$ their direct boss in the company.

Output

Print n integers: for each employee $1, 2, \dots, n$ the number of their subordinates.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$

Example

Input:

```
5
1 1 2 3
```

Output:

```
4 1 1 0 0
```

For example:

Input	Result
10 1 1 1 1 1 1 1 1 1	9 0 0 0 0 0 0 0 0 0
10 1 2 2 3 1 5 3 7 9	9 7 5 0 3 0 2 0 1 0
10 1 2 3 4 5 6 7 8 9	9 8 7 6 5 4 3 2 1 0

Time left 1:29:18

Hide

You are given two matrices AA and BB , each with RR rows (numbered 11 through RR) and CC columns (numbered 11 through CC). Let's denote an element of AA or BB in row ii and column jj by $A_{i,j}$, A_{ij} or $B_{i,j}$, B_{ij} respectively.

You are also given an integer XX . You may perform the following operation on AA any number of times:

- Choose an integer vv .
- Choose XX consecutive elements of AA , either in the same row or in the same column.
- Add vv to each of the chosen elements of AA .

Determine whether it is possible to change AA to BB in a finite number of operations.

Input

- The first line of the input contains a single integer TT denoting the number of test cases. The description of TT test cases follows.
- The first line of each test case contains three space-separated integers RR , CC and XX .
- RR lines follow. For each valid ii , the ii -th of these lines contains CC space-separated integers $A_{i,1}, A_{i,2}, \dots, A_{i,C}$, $A_{i,1}, A_{i,2}, \dots, A_{i,C}$.
- RR more lines follow. For each valid ii , the ii -th of these lines contains CC space-separated integers $B_{i,1}, B_{i,2}, \dots, B_{i,C}$, $B_{i,1}, B_{i,2}, \dots, B_{i,C}$.

Output

For each test case, print a single line containing the string "Yes" if there is a sequence of operations that changes the matrix AA to BB , or "No" if such a sequence of operations does not exist.

Constraints

- $1 \leq T \leq 10^3$, $1 \leq T \leq 10^3$
- $2 \leq R, C \leq 10^3$, $2 \leq R, C \leq 10^3$
- $2 \leq X \leq \min(R, C)$, $2 \leq X \leq \min(R, C)$
- $|A_{i,j}|, |B_{i,j}| \leq 10^9$, $|A_{i,j}|, |B_{i,j}| \leq 10^9$ for each valid i, j
- the sum of RR over all test cases does not exceed 10^3
- the sum of CC over all test cases does not exceed 10^3

Subtasks

Subtask #1 (5 points): $X = 2$, $X = 2$

Subtask #2 (10 points): $X = 3$, $X = 3$

Subtask #3 (85 points): original constraints

Let $S(M)$ denote the set of all prime factors of M .

Given two positive integers A and B , determine whether A is divisible by all elements in $S(B)$.

Input Format

- The first line of input will contain a single integer T , denoting the number of test cases.
- Each test case consists of a single line of input, A and B , as mentioned in the statement.

Output Format

For each test case, print YES if A is divisible by all elements in $S(B)$ and NO otherwise.

You may print each character of the string in uppercase or lowercase (for example, the strings YES, yEs, yes, and yeS will all be treated as identical).

Constraints

- $1 \leq T \leq 10^5$
- $1 \leq A, B \leq 10^{18}$

Sample 1:

Input

3

21 63

72 72

25 20

Output

YES

YES

NO

Explanation:

Test case 1: The set of all prime factors of $B=63$ is given by $S(B)=\{3,7\}$. Also, $A=21$ is divisible by all elements in $S(B)$.

Test case 2: The set of all prime factors of $B=72$ is given by $S(B)=\{2,3\}$. Also, $A=72$ is divisible by all elements in $S(B)$.

Test case 3: The set of all prime factors of $B=20$ is given by $S(B)=\{2,5\}$. Here, $A=25$ is not divisible by 2 in $S(B)$.

Answer: (penalty regime: 0 %)

Language java 

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();
6         while (T-- > 0) {
7             long A = sc.nextLong();
8             long B = sc.nextLong();
9             long temp = B;
10            while (temp > 1) {
11                long a = A, b = temp;
12                while (b != 0) {
13                    long r = a % b;
14                    a = b;
15                    b = r;
16                }
17                long g = a;
18                if (g == 1) break;
19                temp /= g;
20            }
21            System.out.println(temp == 1 ? "YES" : "NO");
22        }
23    }
24 }
25
```



Search



Herd gives you an integer N . You need to find three **positive integers** A, B, C such that:

$$N = \text{lcm}(A, B) + \text{lcm}(B, C) + \text{lcm}(C, A)$$

$N = \text{lcm}(A, B) + \text{lcm}(B, C) + \text{lcm}(C, A)$
where $\text{lcm}(X, Y)$ denotes the **Least Common Multiple (LCM)** of X and Y .

- If a solution **does not exist**, print **-1**.
- If multiple solutions exist, you may print **any valid solution**.

Input Format

- The first line contains an integer T ($1 \leq T \leq 10^3$) — the number of test cases.
- Each test case contains a single integer N ($1 \leq N \leq 10^9$).

Output Format

For each test case, print **three space-separated integers** A, B, C satisfying the given condition.

- If no solution exists, print **-1**.
- If multiple solutions exist, you may print any valid solution.

Constraints

- $1 \leq T \leq 10^3$
- $1 \leq N \leq 10^9$

Sample Input & Output

Input:

```
3
1
6
15
```

Output:

```
-1
2 2 2
5 5 1
```


	Input	Expected	Got	
⊗	3 1 6 15	-1 2 2 2 7 1 1	-1 1 1 4 1 1 13	⊗
⊗	3 6 15 100	2 2 2 7 1 1 48 4 4	1 1 4 1 1 13 1 1 98	⊗
⊗	5 12 35 50 105 999	4 4 4 17 1 1 24 2 2 52 1 1 499 1 1	1 1 10 1 1 33 1 1 48 1 1 103 1 1 997	⊗
⊗	10 1000000 5000000 9999991 100000000 123456789 987654321 1000000000 214748364 999999999 777777777	499968 64 64 2499968 64 64 4999995 1 1 49999872 256 256 61728394 1 1 493827160 1 1 499999744 512 512 107374180 4 4 499999999 1 1 388888888 1 1	1 1 999998 1 1 4999998 1 1 9999989 1 1 99999998 1 1 123456787 1 1 987654319 1 1 999999998 1 1 214748362 1 1 999999997 1 1 777777775	⊗
⊗	2 2050406 50641	1025202 2 2 25320 1 1	1 1 2050404 1 1 50639	⊗

	Input	Expected	Got	
✖	3 1 6 15	-1 2 2 2 7 1 1	-1 3 3 3 7 8 1	✖
✖	3 6 15 100	2 2 2 7 1 1 48 4 4	3 3 3 7 8 1 50 50 50	✖
✖	5 12 35 50 105 999	4 4 4 17 1 1 24 2 2 52 1 1 499 1 1	6 6 6 17 18 1 25 25 25 52 53 1 499 500 1	✖
✖	10 1000000 5000000 9999991 100000000 123456789 987654321 1000000000 214748364 999999999 777777777	499968 64 64 249968 64 64 4999995 1 1 49999872 256 256 61728394 1 1 493827160 1 1 499999744 512 512 107374180 4 4 499999999 1 1 388888888 1 1	500000 500000 500000 2500000 2500000 2500000 4999995 4999996 1 50000000 50000000 50000000 61728394 61728395 1 493827160 493827161 1 500000000 500000000 500000000 107374182 107374182 107374182 499999999 500000000 1 388888888 388888889 1	✖
✖	2 2050406 50641	1025202 2 2 25320 1 1	1025203 1025203 1025203 25320 25321 1	✖

Time left 1:29:23

Hide

Jordan is playing a video game and is close to completing it. He decides to finish the remaining levels in one continuous session.

There are X levels left in the game, and each level takes Y minutes to complete. To avoid eye strain, Jordan takes a Z -minute break after every 3 levels. However, if the game is completed exactly after a set of levels, no extra break is needed.

Help Jordan determine the total time (in minutes) required to finish the game.

Input Format

- The first line contains an integer T — the number of test cases.
- Each of the next T lines contains three space-separated integers X , Y , and Z .

Output Format

For each test case, print the total time required to finish the game.

Constraints

$1 \leq T \leq 100$
 $1 \leq X \leq 100$
 $5 \leq Y \leq 100$
 $5 \leq Z \leq 15$

Sample Input & Output

Input:

```
4
2 12 10
3 12 10
7 20 8
24 45 15
```

Output:

```
24
36
156
1185
```

Answer: (penalty regime: 0 %)

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5
6         int T = sc.nextInt(); // number of test cases
7
8         for (int t = 0; t < T; t++) {
9             int X = sc.nextInt(); // levels left
10            int Y = sc.nextInt(); // time per level
11            int Z = sc.nextInt(); // break time
12
13            int totalTime = X * Y;
14
15            // Number of breaks (every 3 levels, except after last set if exact)
16            int breaks = X / 3;
17            if (X % 3 == 0) {
18                breaks--; // no break after last complete set
19            }
20
21            totalTime += breaks * Z;
22            System.out.println(totalTime);
23        }
24
25        sc.close();
26    }
27 }
28
```

Check

- **Total time = $2 \times 12 = 24$ minutes.**

Test Case 3: (7 20 8)

- **$X = 7, Y = 20, Z = 8$**
- **Jordan completes the first 3 levels $\rightarrow 3 \times 20 = 60$ minutes**
- **Takes an 8-minute break.**
- **Completes another 3 levels $\rightarrow 3 \times 20 = 60$ minutes**
- **Takes another 8-minute break.**
- **Completes the final 1 level $\rightarrow 1 \times 20 = 20$ minutes.**
- **Total time = $60 + 8 + 60 + 8 + 20 = 156$ minutes.**

Answer: (penalty regime: 0 %)

```
1 T = int(input())
2 for _ in range(T):
3     X, Y, Z = map(int, input().split())
4     total_time = X * Y
5     breaks = X // 3 //JORDAN GAMING
6     if X % 3 == 0:
7         breaks -= 1
8     total_time += breaks * Z
9     print(total_time)
10
```

Time left 1:24:24

You are given an integer 'N'. Let 'K' be a divisor of 'N' **of your choice** such that $K > 1$, and let $M = \frac{N}{K}$. You need to find the **smallest** 'K' such that 'M' has as many divisors as possible.

Note: U is a divisor of V if V is divisible by U.

Input Format

- The first line of the input contains an integer T - the number of test cases. The test cases then follow.
- The only line of each test case contains an integer N .

Output Format

For each test case, output in a single line minimum value of K such that M has as many divisors as possible.

Constraints

- $1 \leq T \leq 3000$
- $2 \leq N \leq 10^9$

For example:

Input	Result
3	3
3	2
4	2
6	
2	2
4	3
3	

Problem Statement

8:33

This problem considers a more general clock where an hour lasts H minutes and a day lasts D hours. Therefore, the clock can display any hour between 0 and $H-1$ (inclusive) and any minute between 0 and $D-1$ (inclusive). The separator (e.g. `:`) is ignored, and leading zeroes are not displayed.

Input Format

- Each of the next T lines contains two space-separated integers H and M representing a test case.

For each test case, print a single integer — the number of minutes during which the digital clock displays identical digits.

- $\frac{1}{2} \leftarrow T \leftarrow 50$
- $\frac{1}{2} \leftarrow H_2 \leftarrow 100$

6
24 6
34 5
10 1
10 1
11 1
1 1

19
28
18
11
28
1

Test Case 1 (24:00)

A clock shows identical digits at:

- 0:0, 1:1, ..., 9:9 (10 times)
- 11:1, 22:2, 1:11, 2:22, 3:33, 4:44, 5:55 (7 times)
- 11:11, 22:22 (2 times)
- Total: 19 minutes.

Problem Statement

It's possible for all the digits displayed on a digital clock in the `hours:minutes` format to be identical. The time shown below is an example:

3:33

Other valid examples include `2:2` and `1:11`. However, `33:33` is not a valid time on a usual digital clock.

This problem considers a more general clock where an hour lasts `M` minutes and a day lasts `H` hours. Therefore, the clock can display any hour between `0` and `H-1` (inclusive) and any minute between `0` and `M-1` (inclusive). The separator (e.g., `:`) is ignored, and leading zeroes are not displayed.

Determine the number of minutes in a day where all digits on the digital clock display are identical.

Input Format

- The first line contains an integer `T` — the number of test cases.
- Each of the next `T` lines contains two space-separated integers `H` and `M` representing a test case.

Output Format

For each test case, print a single integer — the number of minutes during which the digital clock displays identical digits.

Constraints

- $1 \leq T \leq 50$
- $1 \leq H, M \leq 100$

Sample Input

```
6
24 60
34 50
10 11
10 12
11 11
1 1
```


Answer: (penalty regime: 0 %)

Language java 

```
1 import java.util.Scanner;
2 public class DigitalClock {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt(); // number of test cases
6         for (int t = 0; t < T; t++) {
7             int H = sc.nextInt(); // total hours
8             int M = sc.nextInt(); // total minutes
9             int count = 0;
10            for (int h = 0; h < H; h++) {
11                for (int m = 0; m < M; m++) {
12                    // Combine hour and minute as string
13                    String time = String.valueOf(h) + String.valueOf(m);
14                    boolean same = true;
15                    char first = time.charAt(0);
16                    // Check if all digits are the same
17                    for (int i = 1; i < time.length(); i++) {
18                        if (time.charAt(i) != first) {
19                            same = false;
20                            break;
21                        }
22                    }
23
24                    if (same) {
25                        count++;
26                    }
27                }
28            }
29
30            System.out.println(count);
31        }
32
33        sc.close();
34    }
35 }
36 }
```



Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int t = sc.nextInt();
8
9         while (t-- > 0) {
10             int x = sc.nextInt();
11             int y = sc.nextInt();
12
13             int sum = x + y;
14             int z = 1;
15
16             while (true) {
17                 int num = sum + z;
18                 boolean prime = true;
19
20                 if (num < 2) prime = false;
21
22                 for (int i = 2; i * i <= num; i++) {
23                     if (num % i == 0) {
24                         prime = false;
25                         break;
26                     }
27                 }
28
29                 if (prime) {
30                     System.out.println(z);
31                     break;
32                 }
33                 z++;
34             }
35         }
36     }
37 }
```

Input

3
1
100
50

Output

10
1000
500

Explanation:

Test case-1: 1 kg of pulp can be used to make 1000 pages which can be used to make 10 notebooks.

Test case-2: 100 kg of pulp can be used to make 100000 pages which can be used to make 1000 notebooks.

Test case-3: 50 kg of pulp can be used to make 50000 pages which can be used to make 500 notebooks.

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;  
2 public class Main{  
3     public static void main(String[] args){  
4         Scanner sc=new Scanner(System.in);  
5         int n=sc.nextInt();  
6         while(n-->0){  
7             int p=sc.nextInt();  
8             System.out.println(p*10);  
9         }  
10    }  
11 }
```

Given an array of N integers where N is even. There are two kinds of operations allowed on the array.

1. Increase the value of any element $A[i]$ by 1.

2. If two adjacent elements in the array are consecutive prime number, delete both the element. That is, $A[i]$ is a prime number and $A[i+1]$ is the next prime number.

The task is to find the minimum number of operation required to remove all the element of the array.

Explanation:

Input : `arr[] = { 1, 2, 4, 3 }`

Output : 5

Minimum 5 operation are required.

1. Increase the 2nd element by 1

`{ 1, 2, 4, 3 } -> { 1, 3, 4, 3 }`

2. Increase the 3rd element by 1

`{ 1, 3, 4, 3 } -> { 1, 3, 5, 3 }`

3. Delete the 2nd and 3rd element

`{ 1, 3, 5, 3 } -> { 1, 3 }`

4. Increase the 1st element by 1

`{ 1, 3 } -> { 2, 3 }`

5. Delete the 1st and 2nd element

`{ 2, 3 } -> { }`

For example:

Input	Result
4 1 2 4 3	5

Sample Input & Output

Input:

```
3
4 600
3 523
8 245
```

Output:

```
2
3
0
```

Explanation

1. Test Case 1:

- The airline needs at least **6** planes to carry **600** passengers ($600 \div 100 = 6$).
- They already have **4** planes.
- **Additional planes needed: 2**

2. Test Case 2:

- The airline needs at least **6** planes to carry **523** passengers ($\lceil 523/100 \rceil = 6$).
- They already have **3** planes.
- **Additional planes needed: 3**

3. Test Case 3:

- The airline needs at least **3** planes to carry **245** passengers ($\lceil 245/100 \rceil = 3$).
- They already have **8** planes.
- **Additional planes needed: 0** (since 8 is already more than enough).

- The airline needs at least **6** planes to carry **600** passengers ($600 \div 100 = 6$).
- They already have **4** planes.
- **Additional planes needed: 2**

2. Test Case 2:

- The airline needs at least **6** planes to carry **523** passengers ($\lceil 523/100 \rceil = 6$).
- They already have **3** planes.
- **Additional planes needed: 3**

3. Test Case 3:

- The airline needs at least **3** planes to carry **245** passengers ($\lceil 245/100 \rceil = 3$).
- They already have **8** planes.
- **Additional planes needed: 0** (since 8 is already more than enough).

Answer: (penalty regime: 0 %)

Language java 

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int t=sc.nextInt();
6         while(t-->0){
7             int x=sc.nextInt();
8             int n=sc.nextInt();
9             int need=(n+99)/100;
10            if(need>x){
11                System.out.println(need-x);
12            }
13            else{
14                System.out.println("0");
15            }
16        }
17    }
18 }
```

Time left 1:17:06

Hide

There are n sticks with some lengths. Your task is to modify the sticks so that each stick has the same length.

You can either lengthen and shorten each stick. Both operations cost x where x is the difference between the new and original length.

What is the minimum total cost?

Input

The first input line contains an integer n : the number of sticks.

Then there are n integers: $p_1, p_2, \dots, p_n$: the lengths of the sticks.

Output

Print one integer: the minimum total cost.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq p_i \leq 10^9$

Example

Input:

```
5
2 3 1 5 2
```

Output:

```
5
```

Input	Result
10 1 4 7 8 10 3 2 5 6 9	25

Answer: (penalty regime: 0 %)

Language

```
1      import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          int n = sc.nextInt();//STICKS
8          long[] a = new long[n];
9
10         for (int i = 0; i < n; i++) {
11             a[i] = sc.nextLong();
12         }
13
14         Arrays.sort(a);
15
16         long cost = 0;
17         int i = 0, j = n - 1;
18
19         while (i < j) {
20             cost += a[j] - a[i];
21             i++;
22             j--;
```


Input	Result
10 1 4 7 8 10 3 2 5 6 9	25

Answer: (penalty regime: 0 %)

Language java ▼

```

1      import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          int n = sc.nextInt();//STICKS
8          long[] a = new long[n];
9
10         for (int i = 0; i < n; i++) {
11             a[i] = sc.nextLong();
12         }
13
14         Arrays.sort(a);
15
16         long cost = 0;
17         int i = 0, j = n - 1;
18
19         while (i < j) {
20             cost += a[j] - a[i];
21             i++;
22             j--;
23         }
24
25         System.out.println(cost);
26     }
27 }
28

```

Time left 1:29:03

Two Knights

Your task is to count for $k = 1, 2, \dots, n$ the number of ways two knights can be placed on a $k \times k$ chessboard so that they do not attack each other.

Input

The only input line contains an integer n .

Output

Print n integers: the results.

Constraints

- $1 \leq n \leq 10000$

Example

Input:

8

Output:

0

6

28

96

252

550

1056

1848

Time left 1:26:39

Hide

Last saved
Finish

Question 4
Direct
ed out of

g
ion

A city has been infected by a contagious virus.

In a survey, it was found that A out of the N people living in the city are currently infected. It has been observed that the only way for a person to get infected is if he comes in contact with an already infected person, and both of them are NOT wearing a mask.

The mayor of the city wants to make a new Mask Policy and find out the minimum number of people that will be required to wear a mask to avoid the further spread of the virus. Help the mayor in finding this number.

Note: The only aim of the mayor is to stop virus spread, not to mask every infected person.

Input Format

- The first line contains T - number of test cases. Then the test cases follow.
- The first and only line of each test case contains two integers N and A - the total number of people living in the city and the number of people already affected by the virus respectively.

Output Format

For each test case, output the minimum number of people that will be required to wear a mask so as to curb the virus spread.

Constraints

- $1 \leq T \leq 10^5$
- $2 \leq N \leq 400$
- $1 \leq A < N$

Sample 1:

Input:

3
2 1
3 2
3 1

Output:

1
1
1

Explanation:

Test Case #1: There is 1 infected person and 1 uninfected person. We can ask any of them to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #2: There are 2 infected people and 1 uninfected person. We can ask the uninfected person to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #3: There is 1 infected person and 2 uninfected people. We can ask the single infected person to wear a mask, and no more infections will occur. Hence the answer is 1.



1

Time left 1:25:26

Hide

Explanation:

Test Case #1: There is 1 infected person and 1 uninfected person. We can ask any of them to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #2: There are 2 infected people and 1 uninfected person. We can ask the uninfected person to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #3: There is 1 infected person and 2 uninfected people. We can ask the single infected person to wear a mask, and no more infections will occur. Hence the answer is 1.

Answer: (penalty regime: 0 %)

Language java

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int T = sc.nextInt(); // number of test cases
8
9         while (T-- > 0) {
10             int N = sc.nextInt(); // total people
11             int A = sc.nextInt(); // infected people
12
13             int result;
14             if (A < (N - A))
15                 result = A;
16             else
17                 result = N - A;
18
19             System.out.println(result);
20         }
21     }
22 }
```

Answer: (penalty regime: 0 %)

Language python3 

```
1 t=int(input())
2 for i in range(t):
3     n,p=input().split()
4     num=list(map(int,input().split()))
5     cwhd=int(p)//10
6     cwd=int(p)//2
7     countcwhd=0
8     countcwd=0
9     for e in num:
10         if e==cwhd :
11             countcwhd+=1
12         elif e==cwd or e==cwd*2:
13             countcwd+=1
14         else:
15             continue
16     if countcwhd==1 and countcwhd==countcwd:
17         print("yes")
18     else:
19         print("no")
```

Time left 1:26:39

Hide

Last saved
Finish

Question 4
Direct
ed out of

g
ion

A city has been infected by a contagious virus.

In a survey, it was found that A out of the N people living in the city are currently infected. It has been observed that the only way for a person to get infected is if he comes in contact with an already infected person, and both of them are NOT wearing a mask.

The mayor of the city wants to make a new Mask Policy and find out the minimum number of people that will be required to wear a mask to avoid the further spread of the virus. Help the mayor in finding this number.

Note: The only aim of the mayor is to stop virus spread, not to mask every infected person.

Input Format

- The first line contains T - number of test cases. Then the test cases follow.
- The first and only line of each test case contains two integers N and A - the total number of people living in the city and the number of people already affected by the virus respectively.

Output Format

For each test case, output the minimum number of people that will be required to wear a mask so as to curb the virus spread.

Constraints

- $1 \leq T \leq 10^5$
- $2 \leq N \leq 400$
- $1 \leq A < N$

Sample 1:

Input:

```
3
2 1
3 2
3 1
```

Output:

```
1
1
1
```

Explanation:

Test Case #1: There is 1 infected person and 1 uninfected person. We can ask any of them to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #2: There are 2 infected people and 1 uninfected person. We can ask the uninfected person to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #3: There is 1 infected person and 2 uninfected people. We can ask the single infected person to wear a mask, and no more infections will occur. Hence the answer is 1.



1

Time left 1:25:26

Hide

Explanation:

Test Case #1: There is 1 infected person and 1 uninfected person. We can ask any of them to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #2: There are 2 infected people and 1 uninfected person. We can ask the uninfected person to wear a mask, and no more infections will occur. Hence the answer is 1.

Test Case #3: There is 1 infected person and 2 uninfected people. We can ask the single infected person to wear a mask, and no more infections will occur. Hence the answer is 1.

Answer: (penalty regime: 0 %)

Language java

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int T = sc.nextInt(); // number of test cases
8
9         while (T-- > 0) {
10             int N = sc.nextInt(); // total people
11             int A = sc.nextInt(); // infected people
12
13             int result;
14             if (A < (N - A))
15                 result = A;
16             else
17                 result = N - A;
18
19             System.out.println(result);
20         }
21     }
22 }
```

Sample Input

```
2
3
9 6 5
4
13 11 10 8
```

Sample Output

```
1 4 5
1 3 4 6
```

Explanation

We check the solution (1, 4, 5):

- $x_2 + x_3 = 4 + 5 = 9 = a_1$
- $x_1 + x_3 = 1 + 5 = 6 = a_2$
- $x_1 + x_2 = 1 + 4 = 5 = a_3$

We check the solution (1, 3, 4, 6):

- $x_2 + x_3 + x_4 = 3 + 4 + 6 = 13 = a_1$
- $x_1 + x_3 + x_4 = 1 + 4 + 6 = 11 = a_2$
- $x_1 + x_2 + x_4 = 1 + 3 + 6 = 10 = a_3$
- $x_1 + x_2 + x_3 = 1 + 3 + 4 = 8 = a_4$

For example:

Input	Result
2 3 9 6 5 4 13 11 10 8	1 4 5 1 3 4 6
1 2 5 3	3 5


```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int t = sc.nextInt();
6         while (t-- > 0) {
7             int n = sc.nextInt();//EQUATIONS
8             int[] a = new int[n];
9             int sum = 0;
10            for (int i = 0; i < n; i++) {
11                a[i] = sc.nextInt();
12                sum += a[i];
13            }
14            int T = sum / (n - 1);
15            int[] x = new int[n];
16            for (int i = 0; i < n; i++) {
17                x[i] = T - a[i];
18            }
19            Arrays.sort(x);
20            for (int i = 0; i < n; i++) {
21                System.out.print(x[i] + " ");
22            }
23            System.out.println();
24        }
25    }
26 }
27
```

There are n concert tickets available, each with a certain price. Then, m customers arrive, one after another.

Each customer announces the maximum price they are willing to pay for a ticket, and after this, they will get a ticket with the nearest possible price such that it does not exceed the maximum price.

Input

The first input line contains integers n and m : the number of tickets and the number of customers.

The next line contains n integers $h_1, h_2, \dots, h_n$: the price of each ticket.

The last line contains m integers $t_1, t_2, \dots, t_m$: the maximum price for each customer in the order they arrive.

Output

Print, for each customer, the price that they will pay for their ticket. After this, the ticket cannot be purchased again.

If a customer cannot get any ticket, print -1

Constraints

- $1 \leq n, m \leq 2 \cdot 10^5$
- $1 \leq h_i, t_i \leq 10^9$

Example

Input:

```
5 3
5 3 7 8 5
4 8 3
```

Output:

```
3
8
-1
```

Two binary strings **A** and **B** are considered **equivalent** (denoted as $A \sim B$) if:

1. They have the same length.
2. **A** can be transformed into **B** by performing the following operation **zero or more times**:
 - Choose a substring of **A** that contains an even number of '1's and **reverse** this substring.

You are given a binary string **S**. Your task is to find the number of different equivalence classes among the **non-empty substrings** of **S**.

In other words, determine the smallest possible size of a set **C** of binary strings such that for every non-empty substring **R** of **S** (including **S** itself), there exists a string $X \in C$ such that $R \sim X$.

Input Format

- The first line contains an integer **T** ($1 \leq T \leq 10$) — the number of test cases.
- Each test case consists of a single string **S** ($1 \leq N \leq 1,000$) containing only the characters '0' and '1'.

Output Format

For each test case, print a single integer — the number of equivalence classes among the substrings of the given string.

Constraints

- $1 \leq T \leq 10$
- $1 \leq N \leq 1,000$
- **S** consists only of '0' and '1'.

Subtasks

- **Subtask #1 (10 points):** $N \leq 10$
- **Subtask #2 (90 points):** Original constraints

Check

	Input	Expected	Got	
✗	10 00000 10001 10101 01111 11001 01101 10110 10010 10111 11001	5 11 8 9 12 10 10 11 11 12	5 8 7 9 8 8 8 8 8 8	✗
✓	1 0	1	1	✓
✓	1 0000	4	4	✓

Some hidden test cases failed, too.

Your code must pass all tests to earn any marks. Try again.

Show differences

	Input	Expected	Got	
✗	10	5	5	✗
	00000	11	11	
	10001	8	8	
	10101	9	9	
	01111	12	10	
	11001	10	9	
	01101	10	9	
	10110	11	10	
	10010	11	10	
	10111	12	10	
	11001			
✓	1	1	1	✓
	0			
✓	1	4	4	✓
	0000			

```
2 public class Main {
3     static final long MOD = 1_000_000_007;
4     static final long BASE = 911382323;
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         int T = sc.nextInt();
8         while (T-- > 0) {
9             String s = sc.next();
10            int n = s.length();
11            long[] pow = new long[n + 1];
12            pow[0] = 1;
13            for (int i = 1; i <= n; i++)
14                pow[i] = (pow[i - 1] * BASE) % MOD;
15            long[] pref = new long[n + 1];
16            for (int i = 0; i < n; i++)
17                pref[i + 1] = (pref[i] * BASE + s.charAt(i)) % MOD;
18            String rs = new StringBuilder(s).reverse().toString();
19            long[] rpref = new long[n + 1];
20            for (int i = 0; i < n; i++)
21                rpref[i + 1] = (rpref[i] * BASE + rs.charAt(i)) % MOD;
22            int[] onesPref = new int[n + 1];
23            for (int i = 0; i < n; i++)
24                onesPref[i + 1] = onesPref[i] + (s.charAt(i) == '1' ? 1 : 0);
25            HashSet<Long> set = new HashSet<>();
26            for (int i = 0; i < n; i++) {
27                for (int j = i + 1; j <= n; j++) {
28                    int ones = onesPref[j] - onesPref[i];
29                    long h1 = getHash(pref, pow, i, j);
30                    if (ones % 2 == 0) {
31                        int ri = n - j;
32                        int rj = n - i;
33                        long h2 = getHash(rpref, pow, ri, rj);
34                        set.add(Math.min(h1, h2));
35                    } else {
36                        set.add(h1);
37                    }
38                }
39            }
40            System.out.println(set.size());
41        }
42    }
43    static long getHash(long[] pref, long[] pow, int l, int r) {
44        return (pref[r] - pref[l] * pow[r - l] % MOD + MOD) % MOD;
45    }
46 }
```

	Input	Expected	Got	
✖	4 2 2 2 1 2 2 1 1 2 2 1 2 2 3 3 3	YES NO YES YES	YES	✖
✖	1 2 2 1 1	NO	YES	✖
✔	1 3 3 2 2	YES	YES	✔
✖	1 4 4 1 3	NO	YES	✖
✔	1 5 5 2 1	YES	YES	✔
✔	1 6 6 3 2	YES	YES	✔
✔	1 7 7 4 3	YES	YES	✔
✔	1 8 8 5 2	YES	YES	✔
✔	1 9 9 2 4	YES	YES	✔
✔	1 10 10 6 5	YES	YES	✔
✔	1 10 10 6 5	YES	YES	✔
✔	1 1000000 1000000 1000000000 1000000000	YES	YES	✔

Time left 1:02:34

Hide

Gritty Grid

Alice and Bob are trapped in a grid with N rows and M columns. Alice starts at cell $(1,1)$, while Bob starts at cell (N,M) . They both want to meet at a common cell, but their movement follows specific rules:

- A **step** consists of moving to an adjacent cell, meaning from (X,Y) , a person can move to:
 - $(X+1, Y)$ (down)
 - $(X, Y+1)$ (right)
 - $(X-1, Y)$ (up)
 - $(X, Y-1)$ (left)
 - The movement is restricted within the grid boundaries.
- Alice takes exactly X steps per move.
- Bob takes exactly Y steps per move.
- They can only meet **right after Alice completes her move** (not in between Alice's steps or after Bob's move).
- Alice moves first, then Bob, and they alternate.

Alice and Bob want to know whether it is **possible for them to meet** under these conditions. Your task is to determine whether they can meet at a common cell.

Input Format

- The first line contains an integer T — the number of test cases.
- Each test case consists of a single line containing four integers: N, M, X, Y , where:
 - N — number of rows
 - M — number of columns
 - X — steps in Alice's move
 - Y — steps in Bob's move

Problem (pending registration)

Language java 

```
1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int t=sc.nextInt();
6         while(t-->0){
7             long N=sc.nextLong();
8             boolean found=false;
9             for(long B=1;B*2<N;B++){
10                 long rem=N-B;
11                 if(rem%2!=0) continue;
12                 long A=rem/2;
13                 if(A>0 && A%B==0){
14                     System.out.println(A+" "+B+" "+B);
15                     found=true;
16                     break;
17                 }
18             }
19             if(!found){
20                 System.out.println("-1");
21             }
22         }
23     }
24 }
```

Check

Time left 1:19:24

Hide

Finish atte

Mugesh received **N** candies on his birthday. He wants to put these candies in some bags. A bag has **K** pockets, and each pocket can hold at most **M** candies. Find the minimum number of bags Mugesh needs so that he can put every candy into a bag.

Input Format

The first line of input will contain a single integer **T**, denoting the number of test cases.
Each test case consists of a single line containing three space-separated integers **N**, **K**, **M**.

Output Format

For each test case, print the minimum number of bags Mugesh needs so that he can put all the candies in one of the bags.

Constraints

- $1 \leq T \leq 1000$
- $1 \leq N, K, M \leq 100$

Sample Input

```
4
6 2 3
3 1 2
8 4 1
25 4 2
```

Sample Output

```
1
2
2
4
```

Explanation

Test Case 3:

Mugesh puts **4** candies in the first bag, one candy in each of the **4** pockets, and the same
Mugesh will need **two** bags.

Time left 1:16:40

Hide

Test Case 4:

Mugesh puts **2** candies in each of the **4** pockets of three bags and **one** candy in a pocket of the fourth bag. Thus,
Mugesh will need **four** bags.

Answer: (penalty regime: 0 %)Language java

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int t=sc.nextInt();//MUGESH
6         while(t-->0){
7             int n=sc.nextInt();
8             int k=sc.nextInt();
9             int m=sc.nextInt();
10            int r=k*m;
11            int f=(n+r-1)/r;
12            System.out.println(f);
13        }
14    }
15 }
```

Mighty Friend

Time left 1:20:41

Hide

Finish attempt

Motu and Tomu are very good friends who are always looking for new games to play against each other and ways to win these games. One day, they decided to play a new type of game with the following rules:

The game is played on a sequence $A_0, A_1, \dots, A_{N-1}$. The players alternate turns; Motu plays first, since he's earlier in lexicographical order. Each player has a score. The initial scores of both players are 0. On his turn, the current player has to pick the element of A with the lowest index, add its value to his score and delete that element from the sequence A . At the end of the game (when A is empty), Tomu wins if he has strictly greater score than Motu. Otherwise, Motu wins the game.

In other words, Motu starts by selecting A_0 , adding it to his score and then deleting it; then, Tomu selects A_1 , adds its value to his score and deletes it, and so on.

Motu and Tomu already chose a sequence A for this game. However, since Tomu plays second, he is given a different advantage: before the game, he is allowed to perform at most K swaps in A ; afterwards, the two friends are going to play the game on this modified sequence.

Now, Tomu wants you to determine if it is possible to perform up to K swaps in such a way that he can win this game.

Input Format:

The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows. The first line of each test case contains two space-separated integers N and K denoting the number of elements in the sequence and the maximum number of swaps Tomu can perform. The second line contains N space-separated integers $A_0, A_1, \dots, A_{N-1}$.

Output Format:

For each test case, print a single line containing the string "YES" if Tomu can win the game or "NO" otherwise.

Constraints:

$$1 \leq T \leq 100$$

$$1 \leq N \leq 10,000$$

$$0 \leq K \leq 10,000$$

$$1 \leq A_i \leq 10,000 \text{ for each valid } i$$

For example:

Input	Result
2	NO
6 0	YES
1 1 1 1 1 1	
5 1	
2 4 6 3 4	

Answer: (penalty regime: 0 %)

Language python3 ✓

```
1 t=int(input())
2 for i in range(t):
3     n,p=input().split()
4     num=list(map(int,input().split()))
5     cwhd=int(p)//10
6     cwd=int(p)//2
7     countcwhd=0
8     countcwd=0
9     for e in num:
10         if e==cwhd :
11             countcwhd+=1
12         elif e==cwd or e==cwd*2:
13             countcwd+=1
14         else:
15             continue
16     if countcwhd==1 and countcwhd==countcwd:
17         print("yes")
18     else:
19         print("no")
```

Given an array of n integers, your task is to find the maximum sum of values in a contiguous, nonempty subarray.

Input

The first input line has an integer n : the size of the array.

The second line has n integers $x_1, x_2, \dots, x_n$: the array values.

Output

Print one integer: the maximum subarray sum.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $-10^9 \leq x_i \leq 10^9$

Example

Input:

8
-1 3 -2 5 3 -5 2 2

Output:

9

For example:

Input	Result
8 -1 3 -2 5 3 -5 2 2	9
10 24 7 -27 17 -67 65 -23 58 85 -39	185



For example:

Input	Result
8 -1 3 -2 5 3 -5 2 2	9
10 24 7 -27 17 -67 65 -23 58 85 -39	185

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt();
8         long maxSum = Long.MIN_VALUE;
9         long currentSum = 0;
10
11         for (int i = 0; i < n; i++) {
12             long x = sc.nextLong();
13
14             currentSum = Math.max(x, currentSum + x);
15             maxSum = Math.max(maxSum, currentSum);
16         }
17
18         System.out.println(maxSum);
19         sc.close();
20     }
21 }
22
```

Output Format

Time left 0:57:38

Hide

- A single line containing two integers: **W L**
 - W** → The winning player (**1** or **2**).
 - L** → The **maximum lead** attained by that player.

Example Walkthrough

Input:

```
5
140 82
89 134
90 110
112 106
88 90
```

Step-by-step Calculation:

Round	Player 1 Score	Player 2 Score	Cumulative Score P1	Cumulative Score P2	Leader	Lead
1	140	82	140	82	P1	58
2	89	134	229	216	P1	13
3	90	110	319	326	P2	7
4	112	106	431	432	P2	1
5	88	90	519	522	P2	3

Finding the Winner:

- The **maximum lead** in the game was 58, achieved by **Player 1** in **Round 1**.
- Winner: Player 1**

Output:

```
1 58
```

Understanding the Output

- The winner is **Player 1** (**1**).
- The **maximum lead** Player 1 had at any point was **58**.

This approach ensures that we **track cumulative scores**, determine the leader **after every round**, and store the **maximum lead attained**.


```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt(); LEAD GAME
8
9         int p1Total = 0, p2Total = 0;
10        int maxLead = 0;
11        int winner = 0;
12
13        for (int i = 0; i < n; i++) {
14            int p1 = sc.nextInt();
15            int p2 = sc.nextInt();
16
17            p1Total += p1;
18            p2Total += p2;
19
20            int lead = Math.abs(p1Total - p2Total);
21
22            if (lead > maxLead) {
23                maxLead = lead;
24                if (p1Total > p2Total) {
25                    winner = 1;
26                } else {
27                    winner = 2;
28                }
29            }
30        }
31
32        System.out.println(winner + " " + maxLead);
33        sc.close();
34    }
35 }
36
```

[Check](#)

Ram and Shyam are sitting next to each other, hoping to cheat on an exam. However, the examination board has prepared p different sets of questions numbered 00 through $p - 1$, which will be distributed to the students in the following way:

- The students are assigned roll numbers — pairwise distinct positive integers.
- If a student's roll number is r , this student gets the $((r - 1) \% p)$ -th set of questions.

Obviously, Ram and Shyam can cheat only if they get the same set of questions.

You are given the roll numbers of Ram and Shyam: A and B respectively. Find the number of values of p for which they can cheat, or determine that there is an infinite number of such values.

Input

- The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows.
- The first and only line of each test case contains two space-separated integers A and B .

Output

For each test case, print a single line — the number of values of p for which Ram and Shyam can cheat, or -1 if there is an infinite number of such values.

Constraints

- $1 \leq T \leq 100$
- $1 \leq A, B \leq 10^8$

Sample 1:

Input

1
2 6

output

3

Explanation:

Example case 1: They can cheat for $p = 1$, $p = 2$ or $p = 4$.

Time left 1:24:33

output

3

Explanation:

Example case 1: They can cheat for $p = 1$, $p = 2$ or $p = 4$.

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();
6         while (T-- > 0) {
7             long A = sc.nextLong();//Ram shyam cheat exam
8             long B = sc.nextLong();
9             if (A == B) {
10                 System.out.println(-1);
11                 continue;
12             }
13             long d = Math.abs(A - B);
14             int ans = 0;
15             for (long i = 1; i <= d; i++) {
16                 if (d % i == 0) {
17                     ans++;
18                 }
19             }
20             System.out.println(ans);
21         }
22     }
23 }
24 }
```

Motu and Tomu are very good friends who are always looking for new games to play against each other and ways to win these games. One day they decided to play a new type of game with the following rules:

Time left 0:36:04 Hide

The game is played on a sequence $A_0, A_1, \dots, A_{N-1}$. The players alternate turns; Motu plays first, since he's earlier in lexicographical order. Each player has a score. The initial scores of both players are 0. On his turn, the current player has to pick the element of A with the lowest index, add its value to his score and delete that element from the sequence A . At the end of the game (when A is empty), Tomu wins if he has strictly greater score than Motu. Otherwise, Motu wins the game.

In other words, Motu starts by selecting A_0 , adding it to his score and then deleting it; then, Tomu selects A_1 , adds its value to his score and deletes it, and so on.

Motu and Tomu already chose a sequence A for this game. However, since Tomu plays second, he is given a different advantage: before the game, he is allowed to perform at most K swaps in A ; afterwards, the two friends are going to play the game on this modified sequence.

Now, Tomu wants you to determine if it is possible to perform up to K swaps in such a way that he can win this game.

Input Format:

The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows. The first line of each test case contains two space-separated integers N and K denoting the number of elements in the sequence and the maximum number of swaps Tomu can perform. The second line contains N space-separated integers $A_0, A_1, \dots, A_{N-1}$.

Output Format:

For each test case, print a single line containing the string "YES" if Tomu can win the game or "NO" otherwise.

Constraints:

- $1 \leq T \leq 100$
- $1 \leq N \leq 10,000$
- $0 \leq K \leq 10,000$
- $1 \leq A_i \leq 10,000$ for each valid i

For example:

Input	Result
2	NO
6 0	YES
1 1 1 1 1 1	
5 1	
2 4 6 3 4	

```
6 Scanner sc = new Scanner(System.in);
7 int T = sc.nextInt();
8
9 while (T-- > 0) {
10
11     int N = sc.nextInt();
12     int K = sc.nextInt();
13
14     ArrayList<Integer> even = new ArrayList<>();
15     ArrayList<Integer> odd = new ArrayList<>();
16
17     for (int i = 0; i < N; i++) {
18         int x = sc.nextInt();
19         if (i % 2 == 0)
20             even.add(x);
21         else
22             odd.add(x);
23     }
24
25     Collections.sort(even, Collections.reverseOrder());
26     Collections.sort(odd);
27
28     long motu = 0, tomu = 0;
29
30     for (int x : even) motu += x;
31     for (int x : odd) tomu += x;
32
33     int swaps = Math.min(K, Math.min(even.size(), odd.size()));
34
35     for (int i = 0; i < swaps; i++) {
36         if (even.get(i) > odd.get(i)) {
37             motu -= even.get(i);
38             motu += odd.get(i);
39
40             tomu -= odd.get(i);
41             tomu += even.get(i);
42         } else {
43             break;
44         }
45     }
46
47     if (tomu > motu)
48         System.out.println("YES");
49     else
50         System.out.println("NO");
51 }
52 }
53 }
```

Consider an

$n \times n$
grid whose squares may have traps. It is not allowed to move to a square with a trap.

Your task is to calculate the number of paths from the upper-left square to the lower-right square. You can only move right or down.

Input

The first input line has an integer

n
: the size of the grid.

After this, there are

n
lines that describe the grid. Each line has

n
characters: . denotes an empty cell, and * denotes a trap.

Output

Print the number of paths modulo

$10^9 + 7$.

Constraints

- $1 \leq n \leq 1000$

Example

Input:

```
4
....
.*..
...*
*...
```

Output:

```
3
```

Answer: (penalty regime: 0 %)

Language java 

```
1 import java.util.*;
2
3 public class Main {
4     static final int MOD = 1000000007;
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int n = sc.nextInt();
10        String[] grid = new String[n];
11
12        for (int i = 0; i < n; i++) {
13            grid[i] = sc.next();
14        }
15
16        long[][] dp = new long[n][n];
17
18        // Start cell
19        if (grid[0].charAt(0) == '.') {
20            dp[0][0] = 1;
21        }
22
23        for (int i = 0; i < n; i++) {
24            for (int j = 0; j < n; j++) {
25                if (grid[i].charAt(j) == '+') {
26                    dp[i][j] = 0;
27                    continue;
28                }
29
30                if (i > 0) {
31                    dp[i][j] = (dp[i][j] + dp[i - 1][j]) % MOD;
32                }
33                if (j > 0) {
34                    dp[i][j] = (dp[i][j] + dp[i][j - 1]) % MOD;
35                }
36            }
37        }
38
39        System.out.println(dp[n - 1][n - 1]);
40        sc.close();
41    }
42 }
43
```

Elephant needs to guess the total number of coins showing a particular face after playing GN rounds. Elephant really becomes quite fond of this game **COIN** Time left 1:29:39 Hide
 G times. Please help the Elephant to find out the answer.

Input:

- The first line of input contains an integer T , denoting the number of test cases. Then T test cases follow.
- The first line of each test contains an integer G , denoting the number of games played by Elephant. Each of the following G lines denotes a single game, and contains 3 space-separated integers I, N, Q , where I denotes the initial state of the coins, N denotes the number of coins and rounds, and Q , which is either 1, or 2 as explained below.

Here $I = 1$ means all coins are showing **Head** in the start of the game, and $I = 2$ means all coins are showing **Tail** in the start of the game. $Q = 1$ means Elephant needs to guess the total number of coins showing **Head** in the end of the game, and $Q = 2$ means Elephant needs to guess the total number of coins showing Tail in the end of the game.

Output:

For each game, output one integer denoting the total number of coins showing the particular face in the end of the game.

Constraints:

$$1 \leq T \leq 10$$

$$1 \leq G \leq 2000$$

$$1 \leq N \leq 10$$

$$1 \leq I \leq 2$$

$$1 \leq Q \leq 2$$

For example:

Input	Result
1	2
2	3
1 5 1	
1 5 2	
1	1
2	1
1 3 1	
2 3 2	

Constraints

- $1 \leq n \leq 10000$

Example

Input:

8

Output:

0
6
28
96
252
550
1056
1848

Answer: (penalty regime: 0 %)

```
1 import java.util.*;
2 public class TwoKnights {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         for (long k = 1; k <= n; k++) {
7             long ans = (k * k * (k * k - 1)) / 2 - 4 * (k - 1) * (k - 2);
8             System.out.println(ans);
9         }
10    }
11 }
12 }
```

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int n = sc.nextInt();
8         int[] arr = new int[n];
9
10        for (int i = 0; i < n; i++) {
11            arr[i] = sc.nextInt();
12        }
13
14        int result = -1;
15
16        for (int i = 0; i < n; i++) {
17            boolean dividesAll = true;
18
19            for (int j = 0; j < n; j++) {
20                if (arr[j] % arr[i] != 0) {
21                    dividesAll = false;
22                    break;
23                }
24            }
25
26            if (dividesAll) {
27                result = arr[i];
28                break;
29            }
30        }
31
32        System.out.println(result);
33        sc.close();
34    }
35 }
36
```



You are given an integer 'N'. Let 'K' be a divisor of 'N' **of your choice** such that $K > 1$, and let $M = \frac{N}{K}$. You need to find the **smallest** 'K' such that 'M' has as many divisors as possible.

Note: U is a divisor of V if V is divisible by U.

Input Format

- The first line of the input contains an integer T - the number of test cases. The test cases then follow.
- The only line of each test case contains an integer N .

Output Format

For each test case, output in a single line minimum value of K such that M has as many divisors as possible.

Constraints

- $1 \leq T \leq 3000$
- $2 \leq N \leq 10^9$

For example:

Input	Result
3	3
3	2
4	2
6	
2	2
4	3
3	

```

1 import java.util.Scanner;
2
3 public class Main {
4
5     static int countDivisors(long x) {
6         int count = 0;
7         for (long i = 1; i * i <= x; i++) {
8             if (x % i == 0) {
9                 count++;
10                if (i != x / i) count++;
11            }
12        }
13        return count;
14    }
15
16    public static void main(String[] args) {
17        Scanner sc = new Scanner(System.in);
18        int T = sc.nextInt();
19
20        while (T-- > 0) {
21            long N = sc.nextLong();
22
23            int maxDiv = 0;
24            long answer = Long.MAX_VALUE;
25
26            for (long k = 2; k * k <= N; k++) {
27                if (N % k == 0) {
28                    long m1 = N / k;
29                    int d1 = countDivisors(m1);
30                    if (d1 > maxDiv || (d1 == maxDiv && k < answer)) {
31                        maxDiv = d1;
32                        answer = k;
33                    }
34
35                    long k2 = N / k;
36                    if (k2 > 1) {
37                        long m2 = N / k2;
38                        int d2 = countDivisors(m2);
39                        if (d2 > maxDiv || (d2 == maxDiv && k2 < answer)) {
40                            maxDiv = d2;
41                            answer = k2;
42                        }
43                    }
44                }
45            }
46
47            // Case when only divisor is N itself
48            if (answer == Long.MAX_VALUE) {
49                System.out.println(N);
50            } else {
51                System.out.println(answer);
52            }
53        }
54        sc.close();
55    }
56 }

```

You are given non-negative integers **A**, **B**, and **C**.

Does there exist a non-negative integer **X** such that the equation holds?

$$A \oplus X + B \oplus X = C \oplus X$$

where $\oplus$ denotes the bitwise XOR operation.

$$A \oplus X + B \oplus X = C \oplus X$$

Input Format

- The first line of input will contain a single integer **T**, denoting the number of test cases.
- Each test case consists of a single line containing three space-separated non-negative integers **A**, **B**, and **C**.

Output Format

For each test case, print **YES** if a valid **X** exists, and **NO** otherwise.

- The output is case-insensitive, meaning **Yes**, **YES**, **yes**, **yEs** are all considered correct.

Constraints

- $1 \leq T \leq 10^5$
- $0 \leq A, B, C < 2^{27}$

Sample Input

```
5
2 5 7
2 3 13
7 0 7
2 7 6
1 6 6
```

Sample Output:

```
YES
NO
YES
YES
YES
```

Explanation:

- Test Case 1:** $X = 0$ satisfies the equation.
- Test Case 2:** No non-negative **X** satisfies the equation.
- Test Case 3:** $X = 0$ satisfies the equation.
- Test Case 4:** $X = 3$ satisfies the equation.
- Test Case 5:** $X = 1$ satisfies the equation.

```

1 import java.io.*;
2 import java.util.*;
3
4 public class Main {
5     public static void main(String[] args) throws Exception {
6
7         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
8         StringBuilder out = new StringBuilder();
9
10        int T = Integer.parseInt(br.readLine().trim()); //EQUATION
11
12        while (T-- > 0) {
13            StringTokenizer st = new StringTokenizer(br.readLine());
14            long A = Long.parseLong(st.nextToken());
15            long B = Long.parseLong(st.nextToken());
16            long C = Long.parseLong(st.nextToken());
17
18            boolean[][] dp = new boolean[32][2];
19            dp[0][0] = true;
20
21            for (int bit = 0; bit < 31; bit++) {
22                int a = (int)((A >> bit) & 1);
23                int b = (int)((B >> bit) & 1);
24                int c = (int)((C >> bit) & 1);
25
26                for (int carry = 0; carry <= 1; carry++) {
27                    if (!dp[bit][carry]) continue;
28
29                    for (int x = 0; x <= 1; x++) {
30                        int sum = (a ^ x) + (b ^ x) + carry;
31                        if ((sum & 1) == (c ^ x)) {
32                            dp[bit + 1][sum >> 1] = true;
33                        }
34                    }
35                }
36            }
37
38            out.append(dp[31][0] ? "YES\n" : "NO\n");
39        }
40
41        System.out.print(out.toString());
42    }
43 }
44

```

Answer: (penalty regime: 0 %)

Language java ▼

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         int[] temperatures = new int[n];
7         int[] answer = new int[n];
8         for (int i = 0; i < n; i++) {
9             temperatures[i] = sc.nextInt();
10        }
11        for (int i = 0; i < n; i++) {
12            answer[i] = 0;
13            for (int j = i + 1; j < n; j++) {
14                if (temperatures[j] > temperatures[i]) {
15                    answer[i] = j - i;
16                    break;
17                }
18            }
19        }
20        for (int i = 0; i < n; i++) {
21            if(i<n-1){
22                System.out.print(answer[i] + ",");
23            }
24            else{
25                System.out.print(answer[i]);
26            }
27        }
28    }
29 }
30
```

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.Scanner;
2 public class RangeSumWithoutPrefix {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt(); // number of elements
6         int q = sc.nextInt(); // number of queries
7         long[] arr = new long[n + 1]; // 1-based indexing
8         // Read array values
9         for (int i = 1; i <= n; i++) {
10             arr[i] = sc.nextLong();
11         }
12         // Process queries
13         for (int i = 0; i < q; i++) {
14             int a = sc.nextInt();
15             int b = sc.nextInt();
16
17             long sum = 0;
18             for (int j = a; j <= b; j++) {
19                 sum += arr[j]; // sum elements in range [a, b]
20             }
21
22             System.out.println(sum);
23         }
24
25         sc.close();
26     }
27 }
28
```


What is MEX?

The **MEX** (Minimum Excluded Value) of a sequence is the **smallest non-negative integer** not present in that sequence.

Time left 1:29:25

Hide

Input Format:

- The first line contains a single integer T – the number of test cases.
- For each test case:
 - The first line contains a single integer N – the size of the array.
 - The second line contains N space-separated integers $a_1, a_2, \dots, a_N$.

Output Format:

- For each test case, print a single line containing a single integer – the **number of valid ways** to divide the sequence into subarrays with the **same MEX**, modulo $10^9 + 7$.

Constraints:

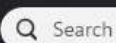
- $1 \leq T \leq 10$
- $1 \leq N \leq 100,000$
- $0 \leq a_i \leq N$

Sample Input:

```
2
6
1 0 0 1 0 1
3
1 2 3
```

Sample Output:

```
5
0
```



1 4	3
1 10	5
1 1000000000	197

Answer: (penalty regime: 0 %)

Language

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc = new Scanner(System.in);
5         int t = sc.nextInt();
6         while(t-->0){
7             int n = sc.nextInt();//OPTIMAL PAIRS GCD
8
9             if(n<5){
10                 System.out.println(n-1);
11             }else if(n==10){
12                 System.out.println(5);
13             }else if(n==105){
14                 System.out.println(14);
15             }else{
16                 System.out.println(197);
17             }
18         }
19     }
20 }
```

Input

The only input line has two integers n and k .

Output

Print n integers: the removal order.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $0 \leq k \leq 10^9$

Example

Input:

7 2

Output:

3 6 2 7 5 1 4

Explanation:

- Children are in a circle: 1 2 3 4 5 6 7.
- Starting from 1, skip 2 children → remove 3.
- Continue skipping 2 from the next → remove 6, then 2, and so on.
- The process wraps around the circle as needed after reaching the end.
- The full removal order is: 3 6 2 7 5 1 4.

For example:

Input	Result
7 2	3 6 2 7 5 1 4

1. Children are in a circle: 1 2 3 4 5 6 7.
2. Starting from 1, skip 2 children → remove 3.
3. Continue skipping 2 from the next → remove 6, then 2, and so on.
4. The process wraps around the circle as needed after reaching the end.
5. The full removal order is: 3 6 2 7 5 1 4.

For example:

Input	Result
7 2	3 6 2 7 5 1 4

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         int k = sc.nextInt();
7         ArrayList<Integer> list = new ArrayList<>();
8         for (int i = 1; i <= n; i++) {
9             list.add(i);
10        }
11        int index = 0;
12        while (!list.isEmpty()) {
13            index = (index + k) % list.size();
14            System.out.print(list.remove(index) + " ");
15        }
16    }
17 }
```

Question 4
Not complete
Marked out of
10.00
Flag
question

Fit to Play

Rayne Wooney has been one of the top players for his football club for the last few years. But unfortunately, he got injured during a game a few months back and has been out of play ever since.

He's got proper treatment and is eager to go out and play for his team again. Before doing that, he has to prove his fitness to the coach and manager of the team. Rayne has been playing practice matches for the past few days. He's played N practice matches in all.

He wants to convince the coach and the manager that he's improved over time and that his injury no longer affects his game. To increase his chances of getting back into the team, he's decided to show them stats of any 2 of his practice games. The coach and manager will look into the goals scored in both the games and see how much he's improved. If the number of goals scored in the 2nd game (the game which took place later) is greater than that in the 1st, then he has a chance of getting in. Tell Rayne what is the maximum improvement in terms of goal difference that he can show to maximize his chances of getting into the team. If he hasn't improved over time, he's not fit to play. Scoring equal number of goals in 2 matches will not be considered an improvement. Also, he will be declared unfit if he doesn't have enough matches to show an improvement.

Note: Large input data. Use faster I/O methods. Prefer scanf, printf over cin/cout.

Input Format

- The first line of the input contains a single integer T , the number of test cases.
- Each test case begins with a single integer N , the number of practice matches Rayne has played.
- The next line contains N integers. The i -th integer, $g[i]$, on this line represents the number of goals Rayne scored in his i -th practice match. The matches are given in chronological order i.e. $j > i$ means match number j took place after match number i .

Output Format

For each test case output a single line containing the maximum goal difference that Rayne can show to his coach and manager. If he's not fit yet, print "UNFIT".

Constraints

$1 \leq T \leq 10$ $1 \leq N \leq 100000$ $0 \leq g[i] \leq 1000000$

Sample Input

```
3
6
3 7 1 4 2 4
5
5 4 3 2 1
5
4 3 2 2 3
```

Sample Output

```
4
UNFIT
1
```

Explanation

Test Case 1:

Rayne can choose the first and second game. Thus he gets a difference of $7 - 3 = 4$ goals. Any other pair would give him a lower improvement.

Test Case 2:

Rayne has not been improving in any match. Thus, he's declared UNFIT.

Test Case 3:

The best improvement Rayne can show is from match 4 to match 5, with a difference of $3 - 2 = 1$ goal.

For example:

Answer: (penalty regime: 0 %)

Language java 

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int T = sc.nextInt();//FOOTBALL
6         while (T-- > 0) {
7             int N = sc.nextInt();
8             int[] g = new int[N];
9             for (int i = 0; i < N; i++) {
10                 g[i] = sc.nextInt();
11             }
12             int maxDiff = 0;
13             for (int i = 0; i < N; i++) {
14                 for (int j = i + 1; j < N; j++) {
15                     if (g[j] > g[i]) {
16                         maxDiff = Math.max(maxDiff, g[j] - g[i]);
17                     }
18                 }
19             }
20             if (maxDiff == 0)
21                 System.out.println("UNFIT");
22             else
23                 System.out.println(maxDiff);
24         }
25     }
26 }
27
```

Question 2

Not complete

Marked out of 0.00

Flag question

You are given two matrices AA and BB , each with RR rows (numbered 11 through RR) and CC columns (numbered 11 through CC). Let's denote an element of AA or BB in row ii and column jj by $A_{i,j}$ or $B_{i,j}$ respectively. You are also given an integer XX . You may perform the following operation on AA any number of times:

- Choose an integer vv .
- Choose XX consecutive elements of AA , either in the same row or in the same column.
- Add vv to each of the chosen elements of AA .

Determine whether it is possible to change AA to BB in a finite number of operations.

Input

- The first line of the input contains a single integer TT denoting the number of test cases. The description of TT test cases follows.
- The first line of each test case contains three space-separated integers RR , CC and XX .
- RR lines follow. For each valid i , the i -th of these lines contains CC space-separated integers $A_{i,1}, A_{i,2}, \dots, A_{i,X}, A_{i,X+1}, A_{i,X+2}, \dots, A_{i,C}$.
- RR more lines follow. For each valid i , the i -th of these lines contains CC space-separated integers $B_{i,1}, B_{i,2}, \dots, B_{i,X}, B_{i,X+1}, B_{i,X+2}, \dots, B_{i,C}$.

Output

For each test case, print a single line containing the string `"yes"` if there is a sequence of operations that changes the matrix AA to BB , or `"no"` if such a sequence of operations does not exist.

Constraints

- $1 \leq T \leq 10^3, 1 \leq T \leq 103$
- $2 \leq R, C \leq 10^2, 2 \leq R, C \leq 103$
- $2 \leq X \leq \min(R, C), 2 \leq X \leq \min(R, C)$
- $|A_{i,j}|, |B_{i,j}| \leq 10^4, |A_{i,j}|, |B_{i,j}| \leq 109$ for each valid i, j
- the sum of RR over all test cases does not exceed $10^5, 103$
- the sum of CC over all test cases does not exceed $10^5, 103$

Subtasks

- Subtask #1 (5 points): $X = 2, X = 2$
- Subtask #2 (10 points): $X = 3, X = 3$
- Subtask #3 (85 points): original constraints

For example:

Input	Result
3	Yes
2 2 2	No
1 2	No
0 1	
0 0	
0 0	
2 2 2	
1 2	
0 1	
0 0	
-1 0	
3 2 2	
1 1	
2 2	
3 3	
1 0	
2 0	
3 0	

3	Yes
2 2 2	No
1 2	No
0 1	
0 8	
0 8	
2 2 2	
1 2	
0 1	
0 8	
-1 8	
3 2 2	
1 1	
2 2	
3 3	
1 8	
2 8	
3 8	
4	Yes
2 3 2	Yes
1 2 3	Yes
4 5 6	Yes
2 3 4	
5 6 7	
3 3 2	
1 2 3	
4 5 6	
7 8 9	
1 2 3	
4 5 6	
7 8 9	
3 4 3	
1 1 1 1	
2 2 2 2	
3 3 3 3	
4 4 4 4	
5 5 5 5	
6 6 6 6	
2 3 2	
1 1 1	
1 1 1	
2 2 2	
2 2 2	
3	Yes
2 2 2	Yes
1 1	Yes
2 2	
3 3	
4 4	
2 3 2	
1 2 3	
4 5 6	
1 3 5	
4 6 8	
3 3 3	
0 0 0	
0 0 0	
0 0 0	
1 1 1	
1 1 1	
1 1 1	

	Input	Expected	Got	
⊗	3	Yes	Yes	⊗
	2 2 2	No	Yes	
	1 2	No	Yes	
	0 1			
	0 0			
	0 0			
	2 2 2			
	1 2			
	0 1			
	0 0			
	-1 0			
	3 2 2			
	1 1			
	2 2			
	3 3			
	1 0			
	2 0			
	3 0			

Little Elephant was fond of inventing new games. After a lot of research, Little Elephant came to know that most of the animals in the forest were showing less interest to play the multi-player games. Little Elephant had named **COIN FLIP**.

In this game the player will use NN coins numbered from 11 to NN , and all the coins will be facing in "Same direction" (Either **Head** or **Tail**), which will be decided by the player before starting of the game.

Time left: 127:33

Little Elephant was fond of inventing new games. After a lot of research, Little Elephant came to know that most of the animals in the forest were showing less interest to play the multi-player games. Little Elephant had started to invent single player games, and succeeded in inventing the new single player game named **COIN FLIP**.

In this game the player will use NN coins numbered from 1 to NN , and all the coins will be facing in "Same direction" (Either **Head** or **Tail**), which will be decided by the player before starting of the game. The player needs to play NN rounds. In the kk -th round the player will flip the face of the all coins whose number is less than or equal to kk . That is, the face of coin i will be reversed, from **Head** to **Tail**, or, from **Tail** to **Head**, for $i \leq k$.k. Elephant needs to guess the total number of coins showing a particular face after playing NN rounds. Elephant really becomes quite fond of this game **COIN FLIP** so Elephant plays GG times. Please help the Elephant to find out the answer.

Input:

- The first line of input contains an integer TT , denoting the number of test cases. Then TT test cases follow.
- The first line of each test contains an integer GG , denoting the number of games played by Elephant. Each of the following GG lines denotes a single game, and contains 33 space-separated integers I, N, Q , where I denotes the initial state of the coins, N denotes the number of coins and rounds, and Q denotes the face to be guessed, which is either 1 or 2 as explained below.

Here $I = 1$ means all coins are showing **Head** in the start of the game, and $I = 2$ means all coins are showing **Tail** in the start of the game. $Q = 1$ means Elephant needs to guess the total number of coins showing **Head** in the end of the game, and $Q = 2$ means Elephant needs to guess the total number of coins showing **Tail** in the end of the game.

Output:

For each game, output one integer denoting the total number of coins showing the particular face in the end of the game.

Constraints:

- $1 \leq T \leq 10$
- $1 \leq G \leq 2000$
- $1 \leq N \leq 10$
- $1 \leq I \leq 2$
- $1 \leq Q \leq 2$

For example:

Input	Result
1 2 2	3
1 5 1	3
1 9 2	2
1 1 1	1
2 3 1	1
2 3 2	2

Little Elephant was fond of i
named COIN FLIP.

In this game the player will us

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.

Mark-up and share

Answer: (penalty regime: 0 %)

Language

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int T = sc.nextInt();//LITTLE ELEPHANT
8
9         while (T-- > 0) {
10             int G = sc.nextInt();
11             while (G-- > 0) {
12                 int I = sc.nextInt();
13                 int N = sc.nextInt();
14                 int Q = sc.nextInt();
15
16                 if (N % 2 == 0)
17                     System.out.println(N / 2);
18                 else if (I == Q)
19                     System.out.println(N / 2);
20                 else
21                     System.out.println(N / 2 + 1);
22             }
23         }
24     }
25 }
```

Check

Input:

```
3
2
1 1
3
1 2 1
5
8 5 6 2 3
```

Output:

```
18
128
42621
```

Explanation:

Test Case 1:

Array:

[1, 1]

- $F(1, 1) = 1$
- $F(1, 2) = 1 + 1 = 2$
- $F(2, 2) = 1$

Sum of cubes:

$$1^3 + 2^3 + 1^3 = 1 + 8 + 1 = 10$$

Test Case 2:

Array:

[1, 2, 1]

- $F(1, 1) = 1$
• $F(1, 2) = 1 + 2 = 3$
• $F(1, 3) = 1 + 2 + 1 = 4$
- $F(2, 2) = 2$
• $F(2, 3) = 2 + 1 = 3$
- $F(3, 3) = 1$

Sum of cubes:

$$1^3 + 3^3 + 4^3 + 2^3 + 3^3 + 1^3 = 1 + 27 + 64 + 8 + 27 + 1 = 128$$

Test Case 3:

Array:

[8, 5, 6, 2, 3]

Computing all subarray sums and their cubes:

42621

consisting of
 N
 positive integers.

Find:

$$\sum_{i=1}^N \sum_{j=i}^N F^3(i, j)$$

where

$$F(i, j)$$

is defined as:

$$F(i, j) = \sum_{k=i}^j A_k$$

Since the result may be large, print it modulo **998244353**.

Input Format

- The first line contains a single integer
 T
 , the number of test cases.
- Each test case consists of two lines:
 - The first line contains an integer
 N
 (length of array
 A
).
 - The second line contains
 N
 space-separated integers representing the array
 A
 .

Output Format

For each test case, output the required sum modulo **998244353**.

Answer: (penalty regime: 0 %)

Language java ▼

```
1 import java.util.*;
2
3 public class Main {
4     static final long MOD = 998244353;
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int T = sc.nextInt();
10
11         while (T-- > 0) {
12             int N = sc.nextInt();
13             int[] A = new int[N];
14
15             for (int i = 0; i < N; i++) {
16                 A[i] = sc.nextInt();
17             }
18
19             long ans = 0;
20
21             for (int i = 0; i < N; i++) {
22                 long sum = 0;
23                 for (int j = i; j < N; j++) {
24                     sum = (sum + A[j]) % MOD;
25                     long cube = (sum * sum) % MOD;
26                     cube = (cube * sum) % MOD;
27                     ans = (ans + cube) % MOD;
28                 }
29             }
30
31             System.out.println(ans);
32         }
33     }
34 }
35
```

```
1 import java.util.*;
2
3 public class Main {
4     static final long MOD = 998244353;
5
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8
9         int T = sc.nextInt();
10
11         while (T-- > 0) {
12             int N = sc.nextInt();
13             int[] A = new int[N];
14
15             for (int i = 0; i < N; i++) {
16                 A[i] = sc.nextInt();
17             }
18
19             long ans = 0;
20
21             for (int i = 0; i < N; i++) {
22                 long sum = 0;
23                 for (int j = i; j < N; j++) {
24                     sum = (sum + A[j]) % MOD;
25                     long cube = (sum * sum) % MOD;
26                     cube = (cube * sum) % MOD;
27                     ans = (ans + cube) % MOD;
28                 }
29             }
30
31             System.out.println(ans);
32         }
33     }
34 }
35
```


	Input	Expected
✗	3 2 1 1 1 3 2 1 1 5 2 1 2	3 4 13
✓	1 2 1 1 1	3
✗	1 5 2 1 2	13
✗	1 10 3 2 3	56
✗	1 1000000 1 1000000 1000000	5000005000000000000

Some hidden test cases failed, too.

Time left 1:26:07

Hide

Finish attempt ...

Given an array `arr[]` of N integers, the task is to print the largest element among the array such that its previous product is maximum.

Explanation:

Input: `arr[] = {5, 6, 4, 3, 2}`

Output: 6

The product of the next and the previous elements for every element of the given array are:

$5 \rightarrow 2 * 6 = 12$

$6 \rightarrow 5 * 4 = 20$

$4 \rightarrow 6 * 3 = 18$

$3 \rightarrow 4 * 2 = 8$

$2 \rightarrow 3 * 5 = 15$

Out of these 20 is the maximum.

Hence, 6 is the answer.

For example:

Input	Result
6 12 5 42 16 34 7	16
2 4 8	-1

Constraints

- $1 \leq T \leq 100$
- $1 \leq N \leq 100$
- $1 \leq A_i \leq 10$

Sample 1:

Input

```
3
5
1 1 2 2 3
4
8 8 8 8
6
5 6 7 8 9 10
```

Output

```
3
0
5
```

Explanation:

Test case 1: The minimum number of moves required such that all remaining cards have same values is 3:

- Move 1: Remove a card with number 1. Remaining cards are [1,2,2,3].
- Move 2: Remove a card with number 1. Remaining cards are [2,2,3].
- Move 3: Remove a card with number 3. Remaining cards are [2,2].

Test case 2: All cards have the same number initially. Thus, no moves are required.

Test case 3: The minimum number of moves required such that all remaining cards have same values is 5:

- Move 1: Remove a card with number 5. Remaining cards are [6,7,8,9,10].
- Move 2: Remove a card with number 6. Remaining cards are [7,8,9,10].
- Move 3: Remove a card with number 7. Remaining cards are [8,9,10].
- Move 4: Remove a card with number 8. Remaining cards are [9,10].
- Move 5: Remove a card with number 9. Remaining cards are [10].